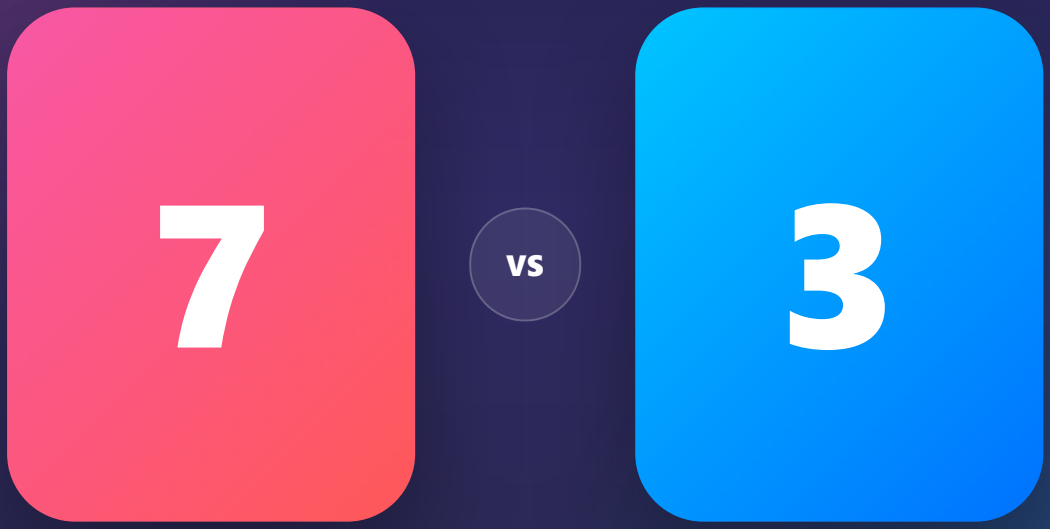
CSE489 · ANDROID DEVELOPMENT

Bigger Number Game

A complete study guide: what to install, a tour of the project, a step-by-step rebuild, and every line of MainActivity.kt explained.



Kotlin · Jetpack Compose · Material 3

October 2026

STUDY GUIDE

Contents

1 Do you need to install a JDK? 4

Android Studio already includes a JDK 4

This project uses JDK 25 4

When you would need a JDK on your system path 4

2 What's in the repository 6

Overview 6

How the game plays 6

Where the code is 6

Things worth knowing 7

3 Rebuild the game step by step 8

Part A: Create the project and look at what the template gives you 8

Step 1: Create the project 8

Step 2: The generated build files (don't change these) 8

Part B: The files you write 13

Step 3: strings.xml, all the text 13

Step 4: colors.xml and themes.xml, no white flash at startup 14

Step 5: Color.kt, the game's colors 14

Step 6: MainActivity.kt, the game, built in stages 15

Step 7: Finish up 29

Testing that it works 30

4 MainActivity.kt line by line 31

Part 1: How the whole game works 31

The file at a glance 31

What's on the screen 32

The key idea: change the data and the screen updates itself 32

What one tap does 33

Part 2: Every line 33

Line 1: Package 33

Lines 3–100: Imports 33

Lines 102–116: MainActivity, where the app starts 36

Lines 118–131: A constant, an enum, and the random helper 37

Lines 133–141: The game's state 38

Lines 143–145: Helpers for vibration and shaking 38

Lines 147–151: newRound 39

Lines 153–173: onGuess, the game's rules 39

Lines 175–180: playAgain 40

Lines 182–211: The background 40

Lines 212–226: Safe area and scrolling 42

Lines 227–268: The three sections 42

Lines 270–320: The pop-ups 44
Lines 322–341: GameHeader 46
Lines 343–390: NumberCard 46
Lines 392–402: VsBadge 48
Lines 404–431: FeedbackMessage 48
Lines 433–496: ScoreCard 49
Lines 498–568: FancyDialog 51
Lines 570–574: The glass helper 53
Lines 576–582: Preview 54
Part 3: Follow some taps through the code 54

CHAPTER 1

Do you need to install a JDK?

> **Short answer: no.** If Android Studio is installed, you already have everything you need to build and run this project.

Android Studio already includes a JDK

Android Studio comes with its own JDK, called the **JetBrains Runtime**. It lives in a `jbr` folder inside the Android Studio install directory, usually:

```
C:\Program Files\Android\Android Studio\jbr
```

Gradle builds inside Android Studio use it by default, so building and running the app from the IDE works without installing anything else.

The Red Hat OpenJDK download page is mostly for Linux servers and containers (it uses commands like `dnf` and `podman`). For Windows, it just points you to Eclipse Temurin.

This project uses JDK 25

The project asks for JDK 25, and Android Studio already provides it.

- `gradle/gradle-daemon-jvm.properties` sets `toolchainVersion=25`, so Gradle itself runs on JDK 25:

gradle/gradle-daemon-jvm.properties · line 12

```
12 toolchainVersion=25
```

- The IDE settings already point to Android Studio's built-in `jbr-25`:

.idea/misc.xml · line 4

```
4   <component name="ProjectRootManager" version="2" languageLevel="JDK_25" default="true" project-jdk-name="jbr-25" project-jdk-type="JavaSDK">
```

If Gradle ever can't find a JDK 25 on your computer, it can download one automatically. The download links are listed in the same `gradle-daemon-jvm.properties` file.

When you would need a JDK on your system path

Only if you want to run commands such as `.\gradlew assembleDebug` or `java -version` from a normal terminal, outside Android Studio. Even then you don't need to install anything new. Point `JAVA_HOME` at the JDK that Android Studio already has, by running this once in PowerShell:

```
[Environment]::SetEnvironmentVariable("JAVA_HOME", "C:\Program Files\Android\Android Studio\jbr", "User")
```

Then open a new terminal so it picks up the change.

To check which JDK Android Studio uses for Gradle, open **Settings → Build, Execution, Deployment → Build Tools → Gradle → Gradle JDK**.

CHAPTER 2

What's in the repository

The repository holds one Android app, the **Bigger Number Game**. It has 49 tracked files, but most of them are created by Android Studio's project template. The game itself lives almost entirely in one Kotlin file.

Overview

Area	Details
Language and UI	Kotlin + Jetpack Compose (Material 3)
Package	`com.example.cse489project`
Android versions	minSdk 24 (Android 7.0); targetSdk and compileSdk 37
Build tools	AGP 9.4.1, Gradle 9.6.0, Kotlin 2.2.10, Compose BOM 2026.02.01
JDK	Gradle runs on JDK 25; the IDE is set to Android Studio's `jbr-25`

How the game plays

- Two different random numbers from 0 to 9 appear on two cards.
- Tap the card with the bigger number. A correct guess earns 1 point; a wrong guess loses 1 point.
- After each guess, a "Correct!" or "Wrong!" message appears.
- Points never go below 0. A wrong guess at 0 points shows a "Go learn math, you dumb!" pop-up instead.
- Reaching 10 points shows a "GENIUS!" pop-up with a **Play Again** button.

To run it, open the project in Android Studio, pick an emulator or a phone connected over USB, and press **Run**. It needs Android 7.0 (API 24) or newer.

Where the code is

Nearly all of the app is in `MainActivity.kt` (582 lines):

- **Game logic** (lines 133–180): two different random numbers from 0 to 9. A correct tap is +1 point and a wrong tap is −1. Points can't go below 0, so a wrong answer at 0 opens the "Dumb" pop-up instead. Reaching 10 opens the "Genius" pop-up. The state is kept with `rememberSaveable`, so it survives screen rotation.
- **UI pieces:** `GameHeader`, `NumberCard` (a gradient card that shrinks when pressed and animates the number in), `VsBadge`, `FeedbackMessage`, `ScoreCard` (animated points and a progress bar to 10), and `FancyDialog` (pops in with a bounce and has a pulsing emoji).
- **Extras:** the cards shake and the phone vibrates on a wrong answer. A `Modifier.glass()` helper gives the frosted-glass look, and the background is a dark gradient with two soft glows.

The other files:

File	What it holds
`ui/theme/Color.kt`	The game's color palette, plus the template's default purple colors

File	What it holds
`ui/theme/Theme.kt` and `ui/theme/Type.kt`	Unchanged from Android Studio's template
`res/values/strings.xml`	All the text the user sees
`res/values/themes.xml`	A dark window background, so the screen doesn't flash white at startup
`ExampleUnitTest.kt` and `ExampleInstrumentedTest.kt`	The template's placeholder tests. Nothing tests the game itself

Things worth knowing

- **You don't need to install a JDK.** The project asks for JDK 25, and Android Studio's built-in `jbr-25` provides it (see Chapter 1).
- **Template leftovers.** The game hard-codes its own colors, so the dynamic color and light/dark color schemes in `Theme.kt` have almost no effect. The purple and teal entries in `colors.xml` aren't used either. They're harmless, but you could delete them.
- **Older dependency versions.** `core-ktx 1.10.1`, `lifecycle-runtime-ktx 2.6.1` and `activity-compose 1.8.0` come from an older template and are behind the rest of the stack. The build still works, but Android Studio will probably suggest upgrading them.
- **App name.** The name in `strings.xml` is still `cse489Project`, which is what appears under the launcher icon.
- **Git history.** When this guide was written (October 2026), the repository had two commits: the initial README, then the whole game added in one commit.

CHAPTER 3

Rebuild the game step by step

Most of the 49 files in the repository are created for you when you make a new project in Android Studio. You only write or change 6 files: `strings.xml`, `colors.xml`, `themes.xml`, `Color.kt`, `MainActivity.kt` and `README.md`. This chapter has two parts:

- **Part A:** create the project, then take a short tour of the files the template generates. You can read them, but you don't need to change them.
- **Part B:** the files you change, in an order where the app still builds after every step.

Code taken from the project shows the file name and the real line numbers, so you can find every piece in the files.

Part A: Create the project and look at what the template gives you

Step 1: Create the project

In Android Studio, choose **File** → **New** → **New Project** → **Empty Activity** (the template with the Compose logo), then fill in:

Field	Value
Name	`cse489Project`
Package name	`com.example.cse489project`
Minimum SDK	API 24 (Android 7.0)
Build configuration language	Kotlin DSL (build.gradle.kts)

Click **Finish** and wait for Gradle sync to finish. Press **Run** once to check that the "Hello Android" screen appears. Then look through the files below.

Step 2: The generated build files (don't change these)

settings.gradle.kts

settings.gradle.kts

```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.android.*")
                includeGroupByRegex("com\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
```

```
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "cse489Project"
include(":app")
```

- **Lines 1–13:** where Gradle downloads **plugins** from, such as the Android build plugin. The `content { }` block tells Gradle to look in Google's repository only for Android, Google and AndroidX packages, which makes downloads faster.
- **Lines 14–16:** the Foojay plugin. If Gradle needs a JDK that isn't on your computer, this plugin lets it download one automatically.
- **Lines 17–23:** where Gradle downloads **libraries** from. `FAIL_ON_PROJECT_REPOS` means modules can't add their own download locations; everything is set here, in one place.
- **Line 25:** the project's name.
- **Line 26:** the project has one module, `app`.

build.gradle.kts (project root)

build.gradle.kts (project root)

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
}
```

This declares the two plugins once for the whole project. `apply false` means "don't use them here"; the `app` module applies them itself. `libs.plugins.…` points into the version catalog, described next.

gradle/libs.versions.toml (the version catalog)

This file is the one place where every library and plugin version is listed.

gradle/libs.versions.toml · excerpt

```
[versions]
agp = "9.4.1"
coreKtx = "1.10.1"
junit = "4.13.2"
junitVersion = "1.1.5"
espressoCore = "3.5.1"
lifecycleRuntimeKtx = "2.6.1"
activityCompose = "1.8.0"
kotlin = "2.2.10"
composeBom = "2026.02.01"
⋮
[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
⋮
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom"
}
androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
⋮
androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
⋮
[plugins]
```

```
29 android-application = { id = "com.android.application", version.ref = "agp" }
30 kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

- `[versions]` gives each version a name, like `agp = "9.4.1"`.
- `[libraries]` describes each library. In build files, the dashes in a name become dots: `androidx-compose-material3` is written `libs.androidx.compose.material3`.
- The Compose libraries have no version of their own. The **BOM** (Bill of Materials, `androidx-compose-bom`) picks versions for all of them that are known to work together.
- `[plugins]` lists the Gradle plugins and their versions.

app/build.gradle.kts (the app module's build file)

app/build.gradle.kts · lines 1–20

```
1  plugins {
2      alias(libs.plugins.android.application)
3      alias(libs.plugins.kotlin.compose)
4  }
5
6  android {
7      namespace = "com.example.cse489project"
8      compileSdk {
9          version = release(37)
10     }
11
12     defaultConfig {
13         applicationId = "com.example.cse489project"
14         minSdk = 24
15         targetSdk = 37
16         versionCode = 1
17         versionName = "1.0"
18
19         testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
20     }
```

- **Lines 1–4:** apply the Android app plugin and the Compose compiler plugin to this module.
- **Line 7:** `namespace` is the package used for the generated `R` class.
- **Lines 8–10:** the Android SDK version used to compile the app (37).
- **Line 13:** `applicationId` is the app's unique ID on a phone and on the Play Store.
- **Line 14:** `minSdk = 24` is the oldest Android version that can install the app (Android 7.0).
- **Line 15:** `targetSdk = 37` is the newest Android version the app has been tested against.
- **Lines 16–17:** the version number used by the Play Store (`versionCode`) and the one shown to users (`versionName`).
- **Line 19:** the test runner used for tests that run on a device.

app/build.gradle.kts · lines 22–37

```
22      buildTypes {
23          release {
24              optimization {
25                  enable = true
26                  packageScope = setOf("androidx.**", "kotlin.**", "kotlinx.**")
27              }
28          }
29      }
30      compileOptions {
31          sourceCompatibility = JavaVersion.VERSION_11
32          targetCompatibility = JavaVersion.VERSION_11
33      }
34      buildFeatures {
35          compose = true
36      }
37  }
```

- **Lines 22–29:** turn on code optimization (R8) for **release** builds. R8 removes unused code and makes the app smaller. It doesn't affect debug builds, which is what you get when you press Run.
- **Lines 30–33:** the Java language level used to compile (Java 11).
- **Lines 34–36:** turn on Jetpack Compose.

app/build.gradle.kts · lines 39–55

```
39  dependencies {
40      implementation(platform(libs.androidx.compose.bom))
41      implementation(libs.androidx.activity.compose)
42      implementation(libs.androidx.compose.material3)
43      implementation(libs.androidx.compose.ui)
44      implementation(libs.androidx.compose.ui.graphics)
45      implementation(libs.androidx.compose.ui.tooling.preview)
46      implementation(libs.androidx.core.ktx)
47      implementation(libs.androidx.lifecycle.runtime.ktx)
48      testImplementation(libs.junit)
49      androidTestImplementation(platform(libs.androidx.compose.bom))
50      androidTestImplementation(libs.androidx.compose.ui.test.junit4)
51      androidTestImplementation(libs.androidx.espresso.core)
52      androidTestImplementation(libs.androidx.junit)
53      debugImplementation(libs.androidx.compose.ui.test.manifest)
54      debugImplementation(libs.androidx.compose.ui.tooling)
55  }
```

- `implementation(...)` adds a library to the app.
- **Line 40:** `platform(...compose.bom)` imports the BOM, which sets the versions of all Compose libraries.
- **Line 41:** `activity-compose` provides `setContent { }`.
- **Line 42:** `material3` provides `Button`, `Text`, `Surface` and the other Material 3 components.
- **Lines 43–44:** the core Compose UI and drawing libraries.
- **Line 45:** the `@Preview` annotation.
- **Lines 46–47:** Kotlin helpers for Android, and lifecycle support.
- **Line 48:** `testImplementation` libraries are only used by unit tests, which run on your PC.
- **Lines 49–52:** `androidTestImplementation` libraries are only used by tests that run on a phone or emulator.
- **Lines 53–54:** `debugImplementation` libraries are only in debug builds. `ui-tooling` draws the `@Preview` in Android Studio.

The other generated files

File	What it does
`gradle.properties`	Gradle settings: 2 GB of memory (`-Xmx2048m`) and the configuration cache, which makes builds faster
`gradle/wrapper/gradle-wrapper.properties`	Which Gradle version to download (9.6.0)
`gradle/gradle-daemon-jvm.properties`	Gradle itself runs on JDK 25. Android Studio's built-in `jbr` provides it, which is why you don't need to install a JDK
`gradlew` , `gradlew.bat` , `gradle-wrapper.jar`	Scripts that let you build from a terminal without installing Gradle
`.gitignore`	Keeps build output and machine-specific files (like `local.properties`) out of Git
`.idea/`	Android Studio's project settings
`app/src/main/keepRules/rules.keep`	Rules for R8. Empty here, which is fine
`res/xml/backup_rules.xml` and `res/xml/data_extraction_rules.xml`	What Android backs up. The game saves no data, so they're left as they are
`res/drawable/ic_launcher_*` and `res/mipmap-*`	The app icon
`ui/theme/Theme.kt`	Creates `Cse489ProjectTheme { }` , which applies Material 3 colors and text styles. The game mostly uses its own colors, but it does use the theme's text styles
`ui/theme/Type.kt`	The text styles (`bodyLarge` , and so on) that the theme uses
`ExampleUnitTest.kt`	A test that runs on your PC. It just checks that `2 + 2 == 4`
`ExampleInstrumentedTest.kt`	A test that runs on a phone or emulator. It checks the package name

AndroidManifest.xml

The manifest is the app's "ID card" for Android.

app/src/main/AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Cse489Project">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.Cse489Project"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

```

```
23                <category android:name="android.intent.category.LAUNCHER" />
24            </intent-filter>
25        </activity>
26    </application>
27
28 </manifest>
```

- **Lines 5–13:** settings for the whole app:
  - `allowBackup`, `dataExtractionRules`, `fullBackupContent`: backup settings, pointing to the two XML files in `res/xml`.
  - `icon` and `roundIcon`: the launcher icons. `@mipmap/...` points to the files in the `mipmap` folders.
  - `label`: the app name shown under the icon. `@string/app_name` points to `strings.xml`.
  - `supportsRtl`: the layout can flip for right-to-left languages.
  - `theme`: the window theme from `themes.xml`, which you'll edit in Step 4.
- **Lines 14–19:** declare `MainActivity`. `.MainActivity` is short for `com.example.cse489project.MainActivity`.
  - `exported="true"` is required on Android 12+ for any activity with an intent filter, so the launcher is allowed to open it.
  - `windowSoftInputMode="adjustResize"` says how the screen resizes when the keyboard opens. This game has no text input, so it doesn't matter here.
- **Lines 20–24:** the `MAIN` + `LAUNCHER` intent filter tells Android to show this activity as an icon in the app drawer.

Part B: The files you write

The order is: **strings** → **colors** → **theme** → **Kotlin colors** → **MainActivity**.

Step 3: strings.xml, all the text

Replace the contents of `res/values/strings.xml` with:

res/values/strings.xml

```
1  <resources>
2      <string name="app_name">cse489Project</string>
3      <string name="game_title">Bigger Number Game!</string>
4      <string name="game_instructions">Press the button of the larger number.  If you get it right, you
   will earn a point!  If you get it wrong, you\'ll lose a point.</string>
5      <string name="which_is_bigger">Which number is bigger?</string>
6      <string name="vs">VS</string>
7      <string name="tap_to_start">Tap the bigger number to start!</string>
8      <string name="correct">✅  Correct!</string>
9      <string name="incorrect">❌  Wrong!</string>
10     <string name="points_label">POINTS</string>
11     <string name="genius_progress">%1$d / %2$d to Genius</string>
12
13     <string name="genius_title">✨ GENIUS! ✨</string>
14     <string name="genius_message">You reached a score of %1$d. Your brain is too big for this game!
   </string>
15     <string name="play_again">Play Again</string>
16
17     <string name="dumb_title">Go learn math, you dumb!</string>
18     <string name="dumb_message">You\'re already at 0 points and you still picked the smaller number.
   </string>
19     <string name="try_again">Try Again</string>
20 </resources>
```

- **Why put text in a resource file?** It keeps text out of the code, and translating the app becomes easy.
- `\'`: an apostrophe has to be escaped in `strings.xml`, or the build fails.
- `%1$d` and `%2$d` are slots for the first and second **d**ecimal numbers. In code you fill them like this: `stringResource(R.string.genius_progress, 3, 10)` gives "3 / 10 to Genius".
- In code, each string is referred to as `R.string.name`, for example `R.string.game_title`.

Step 4: colors.xml and themes.xml, no white flash at startup

In `res/values/colors.xml`, add one line before `</resources>`:

res/values/colors.xml · line 10

```
10      <color name="game_background">#FF0F0C29</color>
```

Then change `res/values/themes.xml` to:

res/values/themes.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3
4      <style name="Theme.Cse489Project" parent="android:Theme.Material.Light.NoActionBar">
5          <!-- Dark window background so there's no white flash before the game draws. -->
6          <item name="android:windowBackground">@color/game_background</item>
7      </style>
8  </resources>
```

Android draws the window background *before* Compose draws anything. If it were white, you'd see a white flash at launch. Here it's set to the same dark navy as the game's background. `NoActionBar` removes the default title bar at the top.

Step 5: Color.kt, the game's colors

Keep the template's `Purple80` and the other default colors, because `Theme.kt` uses them. Below them, add the game's palette. The comments here are only explanations; you can leave them out.

```
// Bigger Number Game palette
val NightTop = Color(0xFF0F0C29)       // background gradient: top
val NightMiddle = Color(0xFF302B63)    //                      middle
val NightBottom = Color(0xFF24243E)    //                      bottom

val LeftCardStart = Color(0xFFF857A6)  // left card: pink → red
val LeftCardEnd = Color(0xFFFF5858)
val RightCardStart = Color(0xFF00C6FF) // right card: cyan → blue
val RightCardEnd = Color(0xFF0072FF)

val GoldStart = Color(0xFFF7971E)      // Genius pop-up + progress bar
val GoldEnd = Color(0xFFFFD200)
val GeniusText = Color(0xFF3E2500)     // dark brown text that's readable on gold

val DumbStart = Color(0xFFE0245E)      // "learn math" pop-up
val DumbEnd = Color(0xFFD7392B)

val CorrectGreen = Color(0xFF16A34A)   // "Correct!" pill
val WrongRed = Color(0xFFE53950)       // "Wrong!" pill
```

`0xFF0F0C29` means **AA RR GG BB**: `FF` is fully opaque, then red `0F`, green `0C`, blue `29`. These are `val`s at the top level of the file, so any file can import them.

Step 6: MainActivity.kt, the game, built in stages

Build this file in stages, and press **Run** after the stages marked ▶. Chapter 4 explains every line of this file in more detail.

About imports

The file needs about 100 imports. The easiest way is to paste the whole import block now. It's lines 3–100 of the finished file, printed below. Imports you haven't used yet only show as grey warnings, not errors. The other way is to type the code and press **Alt+Enter** on each red word to import it.

MainActivity.kt · lines 1–100

```
1  package com.example.cse489project
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.SystemBarStyle
6  import androidx.activity.compose.setContent
7  import androidx.activity.enableEdgeToEdge
8  import androidx.compose.animation.AnimatedContent
9  import androidx.compose.animation.SizeTransform
10 import androidx.compose.animation.core.Animatable
11 import androidx.compose.animation.core.RepeatMode
12 import androidx.compose.animation.core.Spring
13 import androidx.compose.animation.core.animateFloat
14 import androidx.compose.animation.core.animateFloatAsState
15 import androidx.compose.animation.core.infiniteRepeatable
16 import androidx.compose.animation.core.rememberInfiniteTransition
17 import androidx.compose.animation.core.spring
18 import androidx.compose.animation.core.tween
19 import androidx.compose.animation.fadeIn
20 import androidx.compose.animation.fadeOut
21 import androidx.compose.animation.scaleIn
22 import androidx.compose.animation.scaleOut
23 import androidx.compose.animation.slideInVertically
24 import androidx.compose.animation.slideOutVertically
25 import androidx.compose.animation.togetherWith
26 import androidx.compose.foundation.background
27 import androidx.compose.foundation.border
28 import androidx.compose.foundation.interaction.MutableInteractionSource
29 import androidx.compose.foundation.interaction.collectIsPressedAsState
30 import androidx.compose.foundation.layout.Arrangement
31 import androidx.compose.foundation.layout.Box
32 import androidx.compose.foundation.layout.BoxWithConstraints
33 import androidx.compose.foundation.layout.Column
34 import androidx.compose.foundation.layout.ColumnScope
35 import androidx.compose.foundation.layout.Row
36 import androidx.compose.foundation.layout.Spacer
37 import androidx.compose.foundation.layout.fillMaxHeight
38 import androidx.compose.foundation.layout.fillMaxSize
39 import androidx.compose.foundation.layout.fillMaxWidth
40 import androidx.compose.foundation.layout.height
41 import androidx.compose.foundation.layout.heightIn
42 import androidx.compose.foundation.layout.padding
43 import androidx.compose.foundation.layout.safeDrawingPadding
44 import androidx.compose.foundation.layout.size
45 import androidx.compose.foundation.rememberScrollState
46 import androidx.compose.foundation.shape.CircleShape
```

```
47  import androidx.compose.foundation.shape.RoundedCornerShape
48  import androidx.compose.foundation.verticalScroll
49  import androidx.compose.material3.Button
50  import androidx.compose.material3.ButtonDefaults
51  import androidx.compose.material3.LocalContentColor
52  import androidx.compose.material3.MaterialTheme
53  import androidx.compose.material3.Surface
54  import androidx.compose.material3.Text
55  import androidx.compose.runtime.Composable
56  import androidx.compose.runtime.CompositionLocalProvider
57  import androidx.compose.runtime.LaunchedEffect
58  import androidx.compose.runtime.getValue
59  import androidx.compose.runtime.mutableIntStateOf
60  import androidx.compose.runtime.mutableStateOf
61  import androidx.compose.runtime.remember
62  import androidx.compose.runtime.rememberCoroutineScope
63  import androidx.compose.runtime.saveable.rememberSaveable
64  import androidx.compose.runtime.setValue
65  import androidx.compose.ui.Alignment
66  import androidx.compose.ui.Modifier
67  import androidx.compose.ui.draw.clip
68  import androidx.compose.ui.draw.drawBehind
69  import androidx.compose.ui.draw.shadow
70  import androidx.compose.ui.geometry.Offset
71  import androidx.compose.ui.graphics.Brush
72  import androidx.compose.ui.graphics.Color
73  import androidx.compose.ui.graphics.Shape
74  import androidx.compose.ui.graphics.graphicsLayer
75  import androidx.compose.ui.hapticfeedback.HapticFeedbackType
76  import androidx.compose.ui.platform.LocalHapticFeedback
77  import androidx.compose.ui.res.stringResource
78  import androidx.compose.ui.text.font.FontWeight
79  import androidx.compose.ui.text.style.TextAlign
80  import androidx.compose.ui.tooling.preview.Preview
81  import androidx.compose.ui.unit.dp
82  import androidx.compose.ui.unit.sp
83  import androidx.compose.ui.window.Dialog
84  import com.example.cse489project.ui.theme.CorrectGreen
85  import com.example.cse489project.ui.theme.Cse489ProjectTheme
86  import com.example.cse489project.ui.theme.DumbEnd
87  import com.example.cse489project.ui.theme.DumbStart
88  import com.example.cse489project.ui.theme.GeniusText
89  import com.example.cse489project.ui.theme.GoldEnd
90  import com.example.cse489project.ui.theme.GoldStart
91  import com.example.cse489project.ui.theme.LeftCardEnd
92  import com.example.cse489project.ui.theme.LeftCardStart
93  import com.example.cse489project.ui.theme.NightBottom
94  import com.example.cse489project.ui.theme.NightMiddle
95  import com.example.cse489project.ui.theme.NightTop
96  import com.example.cse489project.ui.theme.RightCardEnd
97  import com.example.cse489project.ui.theme.RightCardStart
98  import com.example.cse489project.ui.theme.WrongRed
99  import kotlinx.coroutines.launch
100 import kotlin.random.Random
```

6.1 The Activity ▶

MainActivity.kt · lines 102–116

```
102 class MainActivity : ComponentActivity() {
103     override fun onCreate(savedInstanceState: Bundle?) {
104         super.onCreate(savedInstanceState)
105         // Light status/navigation bar icons, since the game background is dark.
106         enableEdgeToEdge(
107             statusBarStyle = SystemBarStyle.dark(android.graphics.Color.TRANSPARENT),
108             navigationBarStyle = SystemBarStyle.dark(android.graphics.Color.TRANSPARENT)
109         )
110         setContent {
111             Cse489ProjectTheme {
112                 BiggerNumberGame()
113             }
114         }
115     }
116 }
```

- `onCreate` is the first thing Android calls when the screen opens.
- `enableEdgeToEdge` lets the app draw behind the status bar and navigation bar, so the gradient fills the whole screen. `SystemBarStyle.dark(...)` means "the background behind the bars is dark", so Android draws **white** icons (clock, battery) on them.
- `setContent { }` switches from old-style XML layouts to Compose. Everything inside is UI written in Kotlin.
- `Cse489ProjectTheme { }` wraps the UI so `MaterialTheme.typography` works.

For now, add a temporary placeholder below the class, so the app builds:

```
@Composable
fun BiggerNumberGame(modifier: Modifier = Modifier) {
    Text("Hello game")
}
```

`@Composable` marks a function that draws UI. By convention, composables take a `modifier` parameter so the caller can adjust their size or padding. ▶ **Run it**, and you should see "Hello game".

6.2 Constants and the random-number helper

MainActivity.kt · lines 118–131

```
118 /** Score needed to see the "Genius" pop-up. */
119 private const val GENIUS_SCORE = 10
120
121 /** Which pop-up (if any) is showing. */
122 private enum class GameDialog { Genius, Dumb }
123
124 /** Returns a random number from 0 to 9 that is different from [other], so one is always bigger. */
125 private fun randomNumberOtherThan(other: Int): Int {
126     var number = Random.nextInt(10)
127     while (number == other) {
128         number = Random.nextInt(10)
129     }
130     return number
131 }
```

- `GENIUS_SCORE` is a compile-time constant (`const val`). Writing `10` in one place means changing the target later is a one-line edit.

- `GameDialog` has three possible states when combined with `null`: Genius, Dumb, or no pop-up. An enum is safer than a String, because a typo like `"Genuis"` would compile without any error.
- `randomNumberOtherThan` keeps picking until it gets a number different from `other`. The two numbers must be different, or neither one would be "bigger". `Random.nextInt(10)` gives 0 to 9 (10 itself is excluded).

6.3 The glass helper (put it at the bottom of the file)

MainActivity.kt · lines 570–574

```
570 /** Frosted-glass look: a faint white fill with a thin white border. */
571 private fun Modifier.glass(shape: Shape): Modifier = this
572     .clip(shape)
573     .background(Color.White.copy(alpha = 0.08f))
574     .border(1.dp, Color.White.copy(alpha = 0.18f), shape)
```

This is an **extension function**: it adds a new `.glass(...)` method to `Modifier`. Now `Modifier.glass(CircleShape)` gives the frosted-glass look in one call, and it's used twice (the VS badge and the score card).

Modifier order matters. Modifiers are applied top to bottom, so `clip` must come before `background`. Otherwise the background would be drawn as a square.

6.4 GameHeader and VsBadge

MainActivity.kt · lines 322–341

```
322 @Composable
323 private fun GameHeader() {
324     Column(horizontalAlignment = Alignment.CenterHorizontally) {
325         Text(
326             text = stringResource(R.string.game_title),
327             style = MaterialTheme.typography.headlineMedium.copy(
328                 brush = Brush.horizontalGradient(listOf(GoldEnd, LeftCardStart))
329             ),
330             fontWeight = FontWeight.Black,
331             textAlign = TextAlign.Center
332         )
333         Spacer(Modifier.height(8.dp))
334         Text(
335             text = stringResource(R.string.game_instructions),
336             style = MaterialTheme.typography.bodyMedium,
337             color = Color.White.copy(alpha = 0.75f),
338             textAlign = TextAlign.Center
339         )
340     }
341 }
```

- `Column` stacks its children **vertically**. (`Row` places them side by side, and `Box` stacks them on top of each other.)
- `stringResource(R.string.x)` reads the text from `strings.xml`.
- `.copy(brush = ...)` starts from the theme's headline style and changes one thing: the text is filled with a gold-to-pink **gradient** instead of a flat color.
- `Spacer` is empty space. `dp` is a size unit that looks the same on every screen density.

MainActivity.kt · lines 392–402

```kotlin
@Composable
private fun VsBadge() {
    Box(
        modifier = Modifier
            .size(44.dp)
            .glass(CircleShape),
        contentAlignment = Alignment.Center
    ) {
        Text(text = stringResource(R.string.vs), fontSize = 14.sp, fontWeight = FontWeight.Black)
    }
}
```

This is a 44 dp glass circle with "VS" centered inside. `sp` is the unit for text size, and it scales with the user's font-size setting.

6.5 NumberCard, a card you can tap

MainActivity.kt · lines 343–351

```kotlin
/** A big gradient card showing one number. It shrinks a little while pressed. */
@Composable
private fun NumberCard(
    number: Int,
    round: Int,
    colors: List<Color>,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
```

The card doesn't decide what happens when it's tapped. It just calls `onClick`, and the parent decides. This is called **state hoisting**: the parts that draw things don't hold the game's data. That's why the same card can be used for both the left and the right side.

MainActivity.kt · lines 352–355

```kotlin
    val interactionSource = remember { MutableInteractionSource() }
    val pressed by interactionSource.collectIsPressedAsState()
    val scale by animateFloatAsState(if (pressed) 0.92f else 1f, label = "pressScale")
    val shape = RoundedCornerShape(28.dp)
```

- `remember { }` creates something **once** and keeps it every time the UI redraws. (Compose calls the function again whenever something it shows changes.)
- `MutableInteractionSource` reports touch events, and `collectIsPressedAsState()` turns them into a `true` / `false` that says whether a finger is on the card.
- `animateFloatAsState` doesn't jump from 1 to 0.92. It **animates** smoothly between them. That's the "shrink while pressed" effect.
- `by` lets you write `pressed` instead of `pressed.value`.

MainActivity.kt · lines 357–370

```
357     Surface(
358         onClick = onClick,
359         modifier = modifier
360             .height(170.dp)
361             .graphicsLayer {
362                 scaleX = scale
363                 scaleY = scale
364             }
365             .shadow(elevation = 16.dp, shape = shape, spotColor = colors.last()),
366         shape = shape,
367         color = Color.Transparent,
368         contentColor = Color.White,
369         interactionSource = interactionSource
370     ) {
```

- `Surface(onClick = ...)` makes a tappable surface with the ripple effect.
- `graphicsLayer { scaleX = ... }` shrinks how the card is **drawn** without changing the layout, so nothing around it moves. This is the cheapest way to animate size.
- `shadow(spotColor = colors.last())` gives a colored glow (blue under the blue card, red under the red one).
- `color = Color.Transparent` is there because the gradient is drawn by the `Box` inside, and `interactionSource = interactionSource` connects the press detection from above.

MainActivity.kt · lines 371–390

```
371         Box(
372             modifier = Modifier
373                 .fillMaxSize()
374                 .background(Brush.linearGradient(colors)),
375             contentAlignment = Alignment.Center
376         ) {
377             // Keyed on the round too, so the number pops in every round, even if it didn't change.
378             AnimatedContent(
379                 targetState = round to number,
380                 transitionSpec = {
381                     (scaleIn(initialScale = 0.5f) + fadeIn()) togetherWith
382                         (scaleOut(targetScale = 1.5f) + fadeOut())
383                 },
384                 label = "number"
385             ) { (_, value) ->
386                 Text(text = value.toString(), fontSize = 72.sp, fontWeight = FontWeight.Black)
387             }
388         }
389     }
390 }
```

- `AnimatedContent` animates whenever `targetState` changes. Here the new number grows in from half size while the old one grows bigger and fades out.
- **Why `round to number` instead of just `number`?** `a to b` creates a pair. If the left card happened to show 7 two rounds in a row, `number` alone wouldn't change, so there'd be no animation and you wouldn't know a new round started. Including `round` makes the state different every round.
- `{ (_, value) -> }` **unpacks** the pair: `_` ignores the round, and `value` is the number.

6.6 FeedbackMessage, “Correct!” or “Wrong!”

MainActivity.kt · lines 404–431

```
/** Shows "Correct!" or "Wrong!" after each guess, popping in again every round. */
@Composable
private fun FeedbackMessage(round: Int, lastGuessCorrect: Boolean?) {
    AnimatedContent(
        targetState = round to lastGuessCorrect,
        transitionSpec = { (scaleIn(initialScale = 0.7f) + fadeIn()) togetherWith fadeOut() },
        label = "feedback"
    ) { (_, correct) ->
        Box(modifier = Modifier.height(48.dp), contentAlignment = Alignment.Center) {
            if (correct == null) {
                Text(
                    text = stringResource(R.string.tap_to_start),
                    color = Color.White.copy(alpha = 0.7f)
                )
            } else {
                Text(
                    text = stringResource(if (correct) R.string.correct else R.string.incorrect),
                    fontSize = 16.sp,
                    fontWeight = FontWeight.Bold,
                    modifier = Modifier
                        .clip(CircleShape)
                        .background(if (correct) CorrectGreen else WrongRed)
                        .padding(horizontal = 20.dp, vertical = 10.dp)
                )
            }
        }
    }
}
```

- `Boolean?` has three possible values: `null` (no guess yet), `true` and `false`. Each one shows a different message.
- `if` in Kotlin returns a value, so it can pick the string or color right inside the call.
- `clip → background → padding` makes a “pill” shape: rounded green or red behind the text, with space around it. (If padding came first, the space would be outside the pill.)
- The fixed `height(48.dp)` stops the layout from moving when the message changes.
- `round` is in the target state for the same reason as in 6.5: two “Correct!” messages in a row should still animate.

6.7 ScoreCard, points and the progress bar

MainActivity.kt · lines 433–439

```
/** The points display, with a progress bar towards the Genius score. */
@Composable
private fun ScoreCard(points: Int) {
    val progress by animateFloatAsState(
        targetValue = points.coerceIn(0, GENIUS_SCORE) / GENIUS_SCORE.toFloat(),
        label = "progress"
    )
```

Progress is a number from 0.0 to 1.0, so 3 points gives 0.3. `coerceIn` keeps it in that range. `.toFloat()` matters: `3 / 10` with two Ints is **0** (integer division), but `3 / 10f` is 0.3. The bar animates smoothly when progress changes.

MainActivity.kt · lines 441–454

```
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .glass(RoundedCornerShape(24.dp))
            .padding(20.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = stringResource(R.string.points_label),
            fontSize = 12.sp,
            fontWeight = FontWeight.Bold,
            letterSpacing = 4.sp,
            color = Color.White.copy(alpha = 0.7f)
        )
```

A glass card holding a small, widely spaced “POINTS” label.

MainActivity.kt · lines 455–468

```
        // The number slides up when points go up, and down when they go down.
        AnimatedContent(
            targetState = points,
            transitionSpec = {
                if (targetState > initialState) {
                    (slideInVertically { it } + fadeIn()) togetherWith (slideOutVertically { -it } +
fadeOut())
                } else {
                    (slideInVertically { -it } + fadeIn()) togetherWith (slideOutVertically { it } +
fadeOut())
                }.using(SizeTransform(clip = false))
            },
            label = "points"
        ) { value ->
            Text(text = value.toString(), fontSize = 48.sp, fontWeight = FontWeight.Black)
        }
```

- `initialState` is the old value and `targetState` is the new one, so you can pick the direction. When points go **up**, the new number slides in from below (`{ it }` is +height) and the old one leaves upward (`-it`). When points go down, it’s the reverse, like a counter turning.
- `SizeTransform(clip = false)` lets the sliding numbers draw outside the box instead of being cut off.

MainActivity.kt · lines 469–496

```
        Spacer(Modifier.height(8.dp))
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(10.dp)
                .clip(CircleShape)
                .background(Color.White.copy(alpha = 0.15f))
        ) {
            Box(
                modifier = Modifier
                    .fillMaxWidth(progress)
                    .fillMaxHeight()
                    .clip(CircleShape)
                    .background(Brush.horizontalGradient(listOf(GoldStart, GoldEnd)))
            )
        }
        Spacer(Modifier.height(6.dp))
```

```
        Text(
            text = stringResource(
                R.string.genius_progress,
                points.coerceAtMost(GENIUS_SCORE),
                GENIUS_SCORE
            ),
            fontSize = 12.sp,
            color = Color.White.copy(alpha = 0.7f)
        )
    }
}
```

The progress bar is made from two boxes. The outer one is the full-width grey track. The inner one uses `fillMaxWidth(progress)`, meaning "take up `progress` × the width", so at 0.3 it covers 30%. At the end, the `%1$d / %2$d` slots from `strings.xml` are filled in.

6.8 FancyDialog, the pop-up

MainActivity.kt · lines 498–508

```
/** A colorful pop-up that bounces in, with a pulsing emoji on top. */
@Composable
private fun FancyDialog(
    emoji: String,
    title: String,
    message: String,
    gradient: List<Color>,
    textColor: Color,
    onDismissRequest: () -> Unit,
    actions: @Composable ColumnScope.() -> Unit
) {
```

One function is used for **both** pop-ups. The last parameter is a **slot**: the caller passes in its own composable content (a button). `ColumnScope.` means that content is placed inside the dialog's Column.

MainActivity.kt · lines 509–519

```
    Dialog(onDismissRequest = onDismissRequest) {
        val popIn = remember { Animatable(0.6f) }
        LaunchedEffect(Unit) {
            popIn.animateTo(
                targetValue = 1f,
                animationSpec = spring(
                    dampingRatio = Spring.DampingRatioMediumBouncy,
                    stiffness = Spring.StiffnessMediumLow
                )
            )
        }
```

- `Dialog` shows its content in a window above the app with a dimmed background. `onDismissRequest` is called when the user presses Back or taps outside the dialog.
- `Animatable` is a value you animate yourself from code. It starts at 0.6 (60% size).
- `LaunchedEffect(Unit)` runs its block **once**, when the dialog first appears. (`Unit` never changes, so it never runs again.) It's needed because `animateTo` is a `suspend` function: it takes time, so it has to run inside a coroutine.
- A `spring` with a bouncy damping ratio goes slightly past 1.0 and settles back. That's the "boing".

MainActivity.kt · lines 520–525

```
        val pulse by rememberInfiniteTransition(label = "pulse").animateFloat(
            initialValue = 1f,
            targetValue = 1.15f,
            animationSpec = infiniteRepeatable(tween(durationMillis = 700), RepeatMode.Reverse),
            label = "emojiScale"
        )
```

This animation runs forever: 1.0 → 1.15 → 1.0 → … with each half taking 700 ms. It's the emoji's heartbeat.

MainActivity.kt · lines 526–568

```
        val shape = RoundedCornerShape(28.dp)

        CompositionLocalProvider(LocalContentColor provides textColor) {
            Column(
                modifier = Modifier
                    .graphicsLayer {
                        scaleX = popIn.value
                        scaleY = popIn.value
                        alpha = ((popIn.value - 0.6f) / 0.4f).coerceIn(0f, 1f)
                    }
                    .shadow(elevation = 24.dp, shape = shape)
                    .clip(shape)
                    .background(Brush.linearGradient(gradient))
                    .padding(28.dp),
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(
                    text = emoji,
                    fontSize = 72.sp,
                    modifier = Modifier.graphicsLayer {
                        scaleX = pulse
                        scaleY = pulse
                    }
                )
                Spacer(Modifier.height(12.dp))
                Text(
                    text = title,
                    style = MaterialTheme.typography.headlineSmall,
                    fontWeight = FontWeight.Black,
                    textAlign = TextAlign.Center
                )
                Spacer(Modifier.height(8.dp))
                Text(
                    text = message,
                    style = MaterialTheme.typography.bodyLarge,
                    textAlign = TextAlign.Center
                )
                Spacer(Modifier.height(24.dp))
                actions()
            }
        }
    }
}
```

- `CompositionLocalProvider(LocalContentColor provides textColor)` sets the **default text color** for everything inside, so you don't have to write `color =` on each `Text`. That's dark brown on gold and white on red.
- The alpha formula turns the scale range 0.6 → 1.0 into opacity 0 → 1, so the dialog fades in while it grows.

- `actions()` draws whatever button the caller passed in.

6.9 The real BiggerNumberGame ▶

Delete the placeholder from 6.1 and write the real version. It has three parts: **state**, **logic** and **layout**.

(a) State: the game's memory

MainActivity.kt · lines 133–141

```
133 @Composable
134 fun BiggerNumberGame(modifier: Modifier = Modifier) {
135     // rememberSaveable keeps the game state when the screen rotates.
136     var leftNumber by rememberSaveable { mutableIntStateOf(Random.nextInt(10)) }
137     var rightNumber by rememberSaveable { mutableIntStateOf(randomNumberOtherThan(leftNumber)) }
138     var points by rememberSaveable { mutableIntStateOf(0) }
139     var round by rememberSaveable { mutableIntStateOf(0) }
140     var lastGuessCorrect by rememberSaveable { mutableStateOf<Boolean?>(null) }
141     var dialog by rememberSaveable { mutableStateOf<GameDialog?>(null) }
```

- `mutableIntStateOf(...)` is an Int that Compose **watches**. When it changes, the parts of the UI that read it are redrawn automatically. This is the core idea of Compose: you change the data, and the UI updates on its own.
- `remember` would keep the value while the screen redraws, but **rotating the phone** destroys and recreates the Activity, which would reset the game. `rememberSaveable` saves the values and restores them after rotation.
- `mutableStateOf<GameDialog?>(null)` means "which pop-up is showing", and `null` means none.

MainActivity.kt · lines 143–145

```
143     val haptics = LocalHapticFeedback.current
144     val scope = rememberCoroutineScope()
145     val shake = remember { Animatable(0f) }
```

These are the vibration helper, a coroutine scope for running animations from a tap handler, and the shake offset, which starts at 0 (not shaking).

(b) Logic: what happens on a tap

MainActivity.kt · lines 147–151

```
147     fun newRound() {
148         leftNumber = Random.nextInt(10)
149         rightNumber = randomNumberOtherThan(leftNumber)
150         round++
151     }
```

Functions declared *inside* a composable can read and change the state variables above them. `round++` restarts the card and message animations.

MainActivity.kt · lines 153–173

```
153     // Called when the user clicks one of the two number cards.
154     fun onGuess(pickedLeft: Boolean) {
155         val correct = if (pickedLeft) leftNumber > rightNumber else rightNumber > leftNumber
156         lastGuessCorrect = correct
157
158         if (correct) {
159             haptics.performHapticFeedback(HapticFeedbackType.Confirm)
160             points++
161             if (points == GENIUS_SCORE) dialog = GameDialog.Genius
162         } else {
```

```
            haptics.performHapticFeedback(HapticFeedbackType.Reject)
            scope.launch {
                for (offset in listOf(-24f, 24f, -16f, 16f, -8f, 0f)) {
                    shake.animateTo(offset, tween(durationMillis = 50))
                }
            }
            // Points never go below 0. Guessing wrong at 0 shows the "learn math" pop-up instead.
            if (points == 0) dialog = GameDialog.Dumb else points--
        }
        newRound()
    }
```

Step by step:

1. `correct` is true if you tapped the side with the bigger number.
2. **Right:** a short "confirm" vibration, then +1 point. Exactly 10 opens the Genius pop-up.
3. **Wrong:** a "reject" vibration, then the cards shake left and right, getting smaller each time (−24, +24, −16, … back to 0, 50 ms per step). `scope.launch` is needed because `animateTo` is a suspend function and a tap handler isn't a coroutine.
4. `if (points == 0) dialog = GameDialog.Dumb else points--` is the rule that **points can't go below 0**. At 0 you get the pop-up instead of losing a point.
5. Every guess, right or wrong, then starts a new round.

MainActivity.kt · lines 175–180

```
    fun playAgain() {
        points = 0
        lastGuessCorrect = null
        dialog = null
        newRound()
    }
```

This resets everything. Setting `dialog = null` makes the pop-up disappear.

(c) Layout: what's on screen

MainActivity.kt · lines 182–211

```
    CompositionLocalProvider(LocalContentColor provides Color.White) {
        Box(
            modifier = modifier
                .fillMaxSize()
                .background(Brush.verticalGradient(listOf(NightTop, NightMiddle, NightBottom)))
                .drawBehind {
                    // Two soft colored glows behind the content.
                    val pinkCenter = Offset(0f, size.height * 0.2f)
                    val blueCenter = Offset(size.width, size.height * 0.75f)
                    val radius = size.width * 0.8f
                    drawCircle(
                        brush = Brush.radialGradient(
                            listOf(LeftCardStart.copy(alpha = 0.3f), Color.Transparent),
                            center = pinkCenter,
                            radius = radius
                        ),
                        radius = radius,
                        center = pinkCenter
                    )
                    drawCircle(
                        brush = Brush.radialGradient(
                            listOf(RightCardStart.copy(alpha = 0.25f), Color.Transparent),
```

```
                            center = blueCenter,
                            radius = radius
                        ),
                        radius = radius,
                        center = blueCenter
                    )
                }
        ) {
```

- All text inside defaults to **white**.
- The background is a dark gradient from top to bottom.
- `drawBehind { }` lets you draw directly with code. Here it draws two big, soft, see-through glows: pink at the top left and blue at the bottom right. `size` is the box's size in pixels. A `radialGradient` from a color to `Transparent` makes each glow fade at the edges.

MainActivity.kt · lines 212–226

```
            BoxWithConstraints(
                modifier = Modifier
                    .fillMaxSize()
                    .safeDrawingPadding()
            ) {
                // Fills the screen in portrait, and scrolls if it doesn't fit (e.g. landscape).
                Column(
                    modifier = Modifier
                        .fillMaxWidth()
                        .verticalScroll(rememberScrollState())
                        .heightIn(min = maxHeight)
                        .padding(horizontal = 20.dp, vertical = 16.dp),
                    horizontalAlignment = Alignment.CenterHorizontally,
                    verticalArrangement = Arrangement.SpaceBetween
                ) {
```

- `safeDrawingPadding()` is needed because of edge-to-edge from 6.1. It keeps the content from going under the status bar or navigation bar. The background was drawn *outside* this padding, which is why it still fills the whole screen.
- `BoxWithConstraints` gives you `maxHeight`, the available screen height.
- `verticalScroll` and `heightIn(min = maxHeight)` together mean the Column is **at least** as tall as the screen. In portrait, `SpaceBetween` spreads the header, cards and score from top to bottom. In landscape, where the content doesn't fit, it scrolls instead of being cut off.

MainActivity.kt · lines 227–268

```
                    GameHeader()

                    Column(
                        modifier = Modifier.padding(vertical = 24.dp),
                        horizontalAlignment = Alignment.CenterHorizontally
                    ) {
                        Text(
                            text = stringResource(R.string.which_is_bigger),
                            style = MaterialTheme.typography.titleMedium,
                            fontWeight = FontWeight.Bold
                        )
                        Spacer(Modifier.height(20.dp))
                        Row(
                            modifier = Modifier
                                .fillMaxWidth()
                                .graphicsLayer { translationX = shake.value.dp.toPx() },
```

```
                            horizontalArrangement = Arrangement.spacedBy(12.dp),
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            NumberCard(
                                number = leftNumber,
                                round = round,
                                colors = listOf(LeftCardStart, LeftCardEnd),
                                onClick = { onGuess(pickedLeft = true) },
                                modifier = Modifier.weight(1f)
                            )
                            VsBadge()
                            NumberCard(
                                number = rightNumber,
                                round = round,
                                colors = listOf(RightCardStart, RightCardEnd),
                                onClick = { onGuess(pickedLeft = false) },
                                modifier = Modifier.weight(1f)
                            )
                        }
                        Spacer(Modifier.height(20.dp))
                        FeedbackMessage(round = round, lastGuessCorrect = lastGuessCorrect)
                    }

                    ScoreCard(points = points)
                }
            }
```

- There are three sections spread by `SpaceBetween`: the header at the top, the game in the middle, and the score at the bottom.
- `graphicsLayer { translationX = shake.value.dp.toPx() }` moves the whole row sideways by the shake amount. It's 0 most of the time and moves only after a wrong answer.
- `Modifier.weight(1f)` on both cards means they **split the leftover width equally** after the VS badge takes its 44 dp.
- `onClick = { onGuess(pickedLeft = true) }` is where the card's tap (from 6.5) connects to the game logic.

MainActivity.kt · lines 270–320

```
            when (dialog) {
                GameDialog.Genius -> FancyDialog(
                    emoji = "🏆",
                    title = stringResource(R.string.genius_title),
                    message = stringResource(R.string.genius_message, GENIUS_SCORE),
                    gradient = listOf(GoldStart, GoldEnd),
                    textColor = GeniusText,
                    // Back button or tapping outside also starts a new game, same as Play Again.
                    onDismissRequest = { playAgain() }
                ) {
                    Button(
                        onClick = { playAgain() },
                        colors = ButtonDefaults.buttonColors(
                            containerColor = GeniusText,
                            contentColor = Color.White
                        ),
                        modifier = Modifier
                            .fillMaxWidth()
                            .height(52.dp)
                    ) {
                        Text(stringResource(R.string.play_again), fontSize = 16.sp, fontWeight =
    FontWeight.Bold)
                    }
                }
```

```
293
294                 GameDialog.Dumb -> FancyDialog(
295                     emoji = "🥵",
296                     title = stringResource(R.string.dumb_title),
297                     message = stringResource(R.string.dumb_message),
298                     gradient = listOf(DumbStart, DumbEnd),
299                     textColor = Color.White,
300                     onDismissRequest = { dialog = null }
301                 ) {
302                     Button(
303                         onClick = { dialog = null },
304                         colors = ButtonDefaults.buttonColors(
305                             containerColor = Color.White,
306                             contentColor = DumbStart
307                         ),
308                         modifier = Modifier
309                             .fillMaxWidth()
310                             .height(52.dp)
311                     ) {
312                         Text(stringResource(R.string.try_again), fontSize = 16.sp, fontWeight =
    FontWeight.Bold)
313                     }
314                 }
315
316                 null -> Unit
317             }
318         }
319     }
320 }
```

- In Compose, **a dialog is just state**. To show a pop-up, set `dialog = GameDialog.Genius`, and the `when` draws it. To close it, set `dialog = null`.
- The `{ Button(...) }` after `FancyDialog(...)` is the `actions` slot from 6.8, written as a **trailing lambda**.
- **Genius:** Back, tapping outside, and the button all call `playAgain()`. **Dumb:** they only close the pop-up, since you're already at 0 points and keep playing.
- `null -> Unit` means no pop-up, so nothing is drawn. Kotlin requires every possible value to be handled in this `when`.

▶ **Run it.** The full game should work now.

6.10 Preview (optional)

MainActivity.kt · lines 576–582

```
576 @Preview(showBackground = true)
577 @Composable
578 fun BiggerNumberGamePreview() {
579     Cse489ProjectTheme {
580         BiggerNumberGame()
581     }
582 }
```

This shows the screen in Android Studio's **Split** or **Design** view without running an emulator.

Step 7: Finish up

- Write the `README.md` describing the game.
- Commit your work: `git add .` then `git commit -m "Add Bigger Number Game"`.

Testing that it works

1. Tap correctly a few times. You should see a green “Correct!”, the points slide up, and the bar grow.
2. Tap wrong at 0 points. You should see the cards shake and the “Go learn math” pop-up.
3. Reach 10 points. You should see the bouncing Genius pop-up, and **Play Again** resets the score to 0.
4. Rotate the phone in the middle of a game. The score should stay the same (that’s `rememberSaveable`), and in landscape the screen should scroll.

CHAPTER 4

MainActivity.kt line by line

The whole game is in `MainActivity.kt`, which has 582 lines. This chapter has three parts:

- **Part 1** gives the big picture, so the line-by-line part makes sense.
- **Part 2** explains every line.
- **Part 3** follows a few taps through the code from start to finish.

Part 1: How the whole game works

The file at a glance

Lines	Code	Job
1	`package`	Which package the file belongs to
3–100	imports	Where each name used in the file comes from
102–116	`MainActivity`	Starts the app and shows the game
119	`GENIUS_SCORE`	The winning score (10)
122	`GameDialog`	Names for the two pop-ups
125–131	`randomNumberOtherThan`	Picks a number different from the other card's
133–320	`BiggerNumberGame`	**The main part:** the game's data, its rules, and the screen layout
322–341	`GameHeader`	Title and instructions
343–390	`NumberCard`	One number card you can tap
392–402	`VsBadge`	The "VS" circle
404–431	`FeedbackMessage`	"Correct!" / "Wrong!"
433–496	`ScoreCard`	Points and the progress bar
498–568	`FancyDialog`	The pop-up
570–574	`glass`	Helper for the frosted-glass look
576–582	Preview	Shows the screen in Android Studio's design view

What's on the screen

```
MainActivity
└ Cse489ProjectTheme
   └ BiggerNumberGame
      └ Box ............................... dark gradient + pink/blue glows
         ├ BoxWithConstraints .............. keeps content out of the system bars
         │  └ Column (scrollable) .......... spreads 3 sections top → bottom
         │     ├ GameHeader ................ title + instructions        (top)
         │     ├ Column .................... (middle)
         │     │  ├ Text .................... "Which number is bigger?"
         │     │  ├ Row ..................... shakes on a wrong answer
         │     │  │  ├ NumberCard .......... left, pink
         │     │  │  ├ VsBadge
         │     │  │  └ NumberCard .......... right, blue
         │     │  └ FeedbackMessage ........ "Correct!" / "Wrong!"
         │     └ ScoreCard ................. points + progress bar        (bottom)
         └ FancyDialog ..................... only when a pop-up should show
```

The key idea: change the data and the screen updates itself

In Compose you never write "change this label's text to 5". It works like this instead:

1. The game keeps its data in **state variables**.
2. Composable functions read those variables to draw the screen.
3. When a state variable changes, Compose **runs those functions again** and the screen shows the new values. This re-run is called *recomposition*.

So the whole game is **6 state variables**, plus **functions that change them when you tap**:

Variable (line)	Type	Starts as	Meaning
`leftNumber` (136)	Int	random 0–9	Number on the left card
`rightNumber` (137)	Int	random 0–9, never equal to left	Number on the right card
`points` (138)	Int	0	The score
`round` (139)	Int	0	Counts guesses. It's never shown; it only restarts the animations
`lastGuessCorrect` (140)	Boolean?	null	null = no guess yet, true/false = result of the last guess
`dialog` (141)	GameDialog?	null	Which pop-up is open (null = none)

What one tap does

```
You tap a card
 └─ NumberCard's Surface calls onClick ..................... line 358
    └─ { onGuess(pickedLeft = true / false) } ............... lines 250 / 258
       └─ onGuess ........................................... line 154
          ├─ right?  vibrate, points + 1
          │          if points reached 10 → dialog = Genius
          ├─ wrong?  vibrate, shake the cards
          │          if points is 0 → dialog = Dumb (otherwise points − 1)
          └─ newRound(): new numbers, round + 1

State changed → Compose redraws:
   the cards animate in the new numbers, the message pops in, the score slides,
   the bar grows, and the pop-up appears if dialog isn't null

Pop-up buttons:
   Genius → "Play Again" → playAgain(): points = 0, dialog = null, new numbers
   Dumb   → "Try Again"  → dialog = null (just closes it; points stay at 0)
```

Part 2: Every line

I skip lines that only contain `}` or `)`, because they just close the block opened above them, and I skip blank lines too. Lines that start with `//` or `/** */` are comments, which the compiler ignores.

Line 1: Package

MainActivity.kt · line 1

```
1 package com.example.cse489project
```

- **1**: Puts this file in the package `com.example.cse489project`. In the manifest, `.MainActivity` means `com.example.cse489project.MainActivity`, so this has to match. Android generates the `R` class (which holds `R.string.…`) in this same package, which is why the file can use `R` without importing it.

Lines 3–100: Imports

An import tells Kotlin where a name comes from, so you can write `Text` instead of `androidx.compose.material3.Text`. Imports don't run any code. Here's what each one is used for.

Android and the Activity (3–7)

Import	Used for
`Bundle`	Type of the saved data passed to `onCreate` (103)
`ComponentActivity`	Parent class of `MainActivity` (102)
`SystemBarStyle`	Colors of the status bar and navigation bar icons (107–108)
`setContent`	Starts the Compose UI (110)
`enableEdgeToEdge`	Lets the app draw behind the system bars (106)

Animation (8–25)

Import	Used for
`AnimatedContent`	Animates from old content to new content when a value changes (378, 407, 456)
`SizeTransform`	How `AnimatedContent` handles a change in size (463)
`Animatable`	A number you animate from code: the shake (145) and the pop-in (510)
`RepeatMode`	`Reverse` makes the emoji pulse go back and forth (523)
`Spring`	Preset settings for the bounce (515–516)
`animateFloat`	The emoji pulse value (520)
`animateFloatAsState`	A value that moves smoothly to a target: card press (354), progress bar (436)
`infiniteRepeatable`	Repeats an animation forever (523)
`rememberInfiniteTransition`	An animation that never stops (520)
`spring`	A bouncy, physics-based animation (514)
`tween`	An animation with a fixed duration (166, 523)
`fadeIn`, `fadeOut`	Fade-in and fade-out transitions
`scaleIn`, `scaleOut`	Grow and shrink transitions
`slideInVertically`, `slideOutVertically`	Slide up/down transitions (460, 462)
`togetherWith`	Runs the enter and exit transitions at the same time

Layout, drawing, touch (26–48)

Import	Used for
`background`	Fills an area with a color or gradient
`border`	Draws an outline (574)
`MutableInteractionSource`	Stream of touch events for a card (352)
`collectIsPressedAsState`	Turns those events into “is a finger on it?” (353)
`Arrangement`	Spacing of children in a Row or Column (225, 243)
`Box`	Layout that stacks its children on top of each other
`BoxWithConstraints`	A Box that tells its content how much space there is (212)
`Column`, `Row`	Layouts that place children top-to-bottom / left-to-right
`ColumnScope`	Type used for the pop-up’s button slot (507)
`Spacer`	An empty gap
`fillMaxSize`, `fillMaxWidth`, `fillMaxHeight`	Take all the available space / width / height
`height`, `size`	Fixed height / fixed width and height
`heightIn`	Minimum height (222)

Import	Used for
`padding`	Empty space inside the edges
`safeDrawingPadding`	Keeps content away from the status bar, navigation bar and camera cutout (215)
`rememberScrollState`, `verticalScroll`	Makes the screen scrollable (221)
`CircleShape`, `RoundedCornerShape`	Shapes for clipping, borders and shadows

Material 3 components (49–54)

Import	Used for
`Button`, `ButtonDefaults`	The pop-up buttons and their colors (280–285, 302–307)
`LocalContentColor`	Default text color for everything inside (182, 528)
`MaterialTheme`	The theme's text styles, such as `headlineMedium`
`Surface`	The tappable container of each card (357)
`Text`	Shows text

Compose runtime and state (55–64)

Import	Used for
`Composable`	The `@Composable` annotation
`CompositionLocalProvider`	Sets a default value for everything inside it (182, 528)
`LaunchedEffect`	Runs a coroutine when something first appears (511)
`getValue`, `setValue`	Make `by` work, so you can write `points` instead of `points.value`
`mutableIntStateOf`, `mutableStateOf`	Create state variables (136–141)
`remember`	Keeps a value while the screen redraws
`rememberCoroutineScope`	A coroutine scope for the shake (144)
`rememberSaveable`	Like `remember`, but also survives rotating the screen (136–141)

Compose UI (65–83)

Import	Used for
`Alignment`	Centering things
`Modifier`	The chain of size, look and behavior settings
`clip`	Cuts content to a shape
`drawBehind`	Draws custom shapes with code (187)
`shadow`	A drop shadow
`Offset`	An (x, y) point (189–190)
`Brush`	Gradients
`Color`	Compose colors
`Shape`	Type of the `glass` helper's parameter (571)

Import	Used for
`graphicsLayer`	Scales, moves or fades how something is drawn
`HapticFeedbackType`	Kinds of vibration: `Confirm`, `Reject` (159, 163)
`LocalHapticFeedback`	The vibration helper (143)
`stringResource`	Reads text from `strings.xml`
`FontWeight`	`Bold`, and `Black` (extra bold)
`TextAlign`	Centers lines of text
`Preview`	The `@Preview` annotation (576)
`dp`, `sp`	Units: `dp` for sizes, `sp` for text (it scales with the user's font-size setting)
`Dialog`	The pop-up window (509)

Your own theme (84–98): `Cse489ProjectTheme` comes from `Theme.kt`. All the colors (`NightTop`, `LeftCardStart`, `GoldEnd`, and so on) come from `Color.kt`.

Kotlin (99–100): `launch` starts a coroutine (164). `Random` makes random numbers.

Lines 102–116: MainActivity, where the app starts

MainActivity.kt · lines 102–116

```
102 class MainActivity : ComponentActivity() {
103     override fun onCreate(savedInstanceState: Bundle?) {
104         super.onCreate(savedInstanceState)
105         // Light status/navigation bar icons, since the game background is dark.
106         enableEdgeToEdge(
107             statusBarStyle = SystemBarStyle.dark(android.graphics.Color.TRANSPARENT),
108             navigationBarStyle = SystemBarStyle.dark(android.graphics.Color.TRANSPARENT)
109         )
110         setContent {
111             Cse489ProjectTheme {
112                 BiggerNumberGame()
113             }
114         }
115     }
116 }
```

- **102**: Declares the class `MainActivity`. An *Activity* is one screen of an Android app. `: ComponentActivity()` means it extends `ComponentActivity`, the kind of Activity that supports Compose. The manifest points to this class, so tapping the app icon opens it.
- **103**: Android calls `onCreate` once, when the screen is created. `override` means "replace the parent class's version". `savedInstanceState: Bundle?` holds saved data when the screen is re-created, for example after rotation. The `?` means it can be `null`, which it is on a fresh launch.
- **104**: Runs the parent class's `onCreate` first. This is required; Android throws an exception if you skip it.
- **106**: Makes the app draw behind the status bar (top) and the navigation bar (bottom), so the gradient fills the whole screen. Android 15 and newer do this anyway. The call makes older versions behave the same way, and it sets the icon colors.
- **107**: `SystemBarStyle.dark(...)` means "the area behind the bar is dark", so Android draws **white** icons (clock, battery). The value inside is a *scrim*, a tint behind the bar, and `TRANSPARENT` means no tint. It's written as

`android.graphics.Color` because the short name `Color` already means Compose's `Color` (line 72), which is a different class.

- **108**: The same for the navigation bar at the bottom.
- **110**: `setContent { }`: everything inside is the screen, written in Compose instead of an XML layout.
- **111**: Wraps the screen in the app's theme (from `Theme.kt`), so `MaterialTheme.typography` gives the right text styles.
- **112**: Starts the game. Everything else in the file runs from here.

Lines 118–131: A constant, an enum, and the random helper

MainActivity.kt · lines 118–131

```
118 /** Score needed to see the "Genius" pop-up. */
119 private const val GENIUS_SCORE = 10
120
121 /** Which pop-up (if any) is showing. */
122 private enum class GameDialog { Genius, Dumb }
123
124 /** Returns a random number from 0 to 9 that is different from [other], so one is always bigger. */
125 private fun randomNumberOtherThan(other: Int): Int {
126     var number = Random.nextInt(10)
127     while (number == other) {
128         number = Random.nextInt(10)
129     }
130     return number
131 }
```

- **119**: The winning score.
  - `val` means it can't be changed.
  - `const` means it's fixed when the app is compiled.
  - `private` means only this file can use it.
  - It's written outside any class or function ("top level").
  - It's used in four places: the win check (161), the Genius message (274), and the score card's bar and text (437, 489–490). Change it to `5` and the whole game follows.
- **122**: An `enum` is a type with a fixed list of values, here `Genius` and `Dumb`. Together with `null` (line 141), that gives three states for the pop-up: none, Genius or Dumb. A typo like `GameDialog.Genuis` is a compile error, which wouldn't happen if you used a plain String.
- **125**: A normal function. It isn't `@Composable`, so it draws nothing. It takes an Int called `other` and returns an Int.
- **126**: `var` is a variable that can change. `Random.nextInt(10)` gives a number from 0 to 9 (10 itself is excluded).
- **127–128**: If the number equals `other`, pick again, as many times as needed.
- **130**: Returns a number that's guaranteed to be different. This matters because if both cards showed the same number, neither would be bigger, and every tap would count as wrong.

Lines 133–141: The game's state

MainActivity.kt · lines 133–141

```
133 @Composable
134 fun BiggerNumberGame(modifier: Modifier = Modifier) {
135     // rememberSaveable keeps the game state when the screen rotates.
136     var leftNumber by rememberSaveable { mutableIntStateOf(Random.nextInt(10)) }
137     var rightNumber by rememberSaveable { mutableIntStateOf(randomNumberOtherThan(leftNumber)) }
138     var points by rememberSaveable { mutableIntStateOf(0) }
139     var round by rememberSaveable { mutableIntStateOf(0) }
140     var lastGuessCorrect by rememberSaveable { mutableStateOf<Boolean?>(null) }
141     var dialog by rememberSaveable { mutableStateOf<GameDialog?>(null) }
```

- **133**: `@Composable` marks a function that builds UI. Composables can only be called from other composables, and the block inside `setContent` counts as one. By convention, their names start with a capital letter.
- **134**: `modifier: Modifier = Modifier` is an optional parameter. The default `Modifier` means "no changes". Composables usually accept one so the caller can change their size or padding. Nothing passes one here, so the default is used.

Every state line has three parts. Take line 138, `var points by rememberSaveable { mutableIntStateOf(0) }`:

- `mutableIntStateOf(0)` creates a **state box** holding 0. Compose watches it, and when its value changes, every composable that read it is redrawn.
- `rememberSaveable { … }`: Compose calls `BiggerNumberGame` again on every recomposition. Without `remember`, each call would create a new box, and the points would reset to 0 every time the screen redraws. `remember` keeps the same box between calls. `rememberSaveable` also saves it when Android re-creates the Activity, which happens when you rotate the phone.
- `var points by …`: `by` lets you write `points` and `points = 5` instead of `points.value` and `points.value = 5`.

Line by line:

- **136**: The left number, random 0–9 at the start. The `{ }` block only runs the first time. After that, the remembered value is used.
- **137**: The right number, random and different from `leftNumber`.
- **138**: The score, which starts at 0.
- **139**: The round counter. It starts at 0 and goes up by 1 after every guess. It's never shown on screen; its only job is to make the animations play every round (explained at line 379).
- **140**: `mutableStateOf` is the general version, for types other than Int. `<Boolean?>` names the type, because from `null` alone Kotlin can't tell which type you mean. `null` means no guess yet.
- **141**: Which pop-up is open, where `null` means none. Setting this to `GameDialog.Genius` is exactly what opens the Genius pop-up.

`mutableIntStateOf` is a version of `mutableStateOf` made just for Int, and it's slightly faster.

Lines 143–145: Helpers for vibration and shaking

MainActivity.kt · lines 143–145

```
143     val haptics = LocalHapticFeedback.current
144     val scope = rememberCoroutineScope()
145     val shake = remember { Animatable(0f) }
```

- **143**: Gets the phone's vibration helper. `LocalHapticFeedback` is a *CompositionLocal*: a value that Compose gives to every composable without passing it as a parameter. `.current` reads it.
- **144**: A coroutine scope. Coroutines are Kotlin's way to run code that takes time, like a 300 ms animation, without freezing the app. `onGuess` is a normal function, so it needs this scope to start the shake. The scope is cancelled automatically if the game leaves the screen.
- **145**: The shake position, in dp. `Animatable(0f)` holds a Float you can animate from code (the `f` means Float), and 0 means "not moved". Plain `remember` is enough here, because there's no need to restore a shake after rotation.

Lines 147–151: newRound

MainActivity.kt · lines 147–151

```
147     fun newRound() {
148         leftNumber = Random.nextInt(10)
149         rightNumber = randomNumberOtherThan(leftNumber)
150         round++
151     }
```

- **147**: A function declared *inside* another function. Kotlin allows this, and the inner function can read and change the state variables above it.
- **148**: A new left number. Because `leftNumber` is state, this assignment also tells Compose to redraw.
- **149**: A new right number, different from the left one.
- **150**: `round++` adds 1. Even if the new numbers happen to equal the old ones, `round` changes, so the cards still animate.

Lines 153–173: onGuess, the game's rules

MainActivity.kt · lines 153–173

```
153     // Called when the user clicks one of the two number cards.
154     fun onGuess(pickedLeft: Boolean) {
155         val correct = if (pickedLeft) leftNumber > rightNumber else rightNumber > leftNumber
156         lastGuessCorrect = correct
157 
158         if (correct) {
159             haptics.performHapticFeedback(HapticFeedbackType.Confirm)
160             points++
161             if (points == GENIUS_SCORE) dialog = GameDialog.Genius
162         } else {
163             haptics.performHapticFeedback(HapticFeedbackType.Reject)
164             scope.launch {
165                 for (offset in listOf(-24f, 24f, -16f, 16f, -8f, 0f)) {
166                     shake.animateTo(offset, tween(durationMillis = 50))
167                 }
168             }
169             // Points never go below 0. Guessing wrong at 0 shows the "learn math" pop-up instead.
170             if (points == 0) dialog = GameDialog.Dumb else points--
171         }
172         newRound()
173     }
```

- **154**: `pickedLeft` is `true` when the left card was tapped and `false` for the right one.
- **155**: In Kotlin, `if` can return a value. If the left card was tapped, the guess is correct when left > right. If the right card was tapped, it's correct when right > left. `val` means this result won't change.

- **156**: Saves the result, so `FeedbackMessage` shows “Correct!” or “Wrong!”.
- **158**: The branch for a correct answer.
- **159**: A short “success” vibration.
- **160**: +1 point. `ScoreCard` sees the new value and animates.
- **161**: If the score is now exactly 10, open the Genius pop-up. Checking with `==` is safe because points only ever go up by 1 at a time.
- **162**: `else` means the answer was wrong.
- **163**: An “error” vibration.
- **164**: `scope.launch { }` starts a coroutine for the shake.
  - It’s needed because `animateTo` (line 166) is a *suspend* function: it waits for the animation to finish, and suspend functions can only run inside a coroutine.
  - `launch` returns right away, so the rest of `onGuess` runs without waiting for the shake to end.
- **165**: Loops through 6 positions, in dp: left 24, right 24, left 16, right 16, left 8, center. The movement gets smaller each time, like a real wobble.
- **166**: Moves from the current position to `offset` in 50 ms and waits until it gets there, then the loop continues.
  - 6 × 50 ms = 300 ms in total.
  - The Row at line 242 reads `shake.value`, which is what makes the cards move.
  - If you tap wrong again during a shake, the new `animateTo` cancels the old one, so the newest shake wins.
- **170**: The “never below 0” rule. At 0 points it opens the Dumb pop-up instead of subtracting. Otherwise it subtracts 1.
- **172**: Every guess starts a new round, right or wrong. This also happens when a pop-up opens, so the numbers behind the pop-up are already new.

Lines 175–180: playAgain

MainActivity.kt · lines 175–180

```
175     fun playAgain() {
176         points = 0
177         lastGuessCorrect = null
178         dialog = null
179         newRound()
180     }
```

- **176**: Score back to 0. The number on the score card slides *down*, because 10 → 0 is a decrease.
- **177**: The message goes back to “Tap the bigger number to start!”.
- **178**: Closes the pop-up, because the `when` at line 270 now draws nothing.
- **179**: New numbers.

Only the Genius pop-up uses this function.

Lines 182–211: The background

From here to line 320 is the layout, meaning what you see. Everything above was data and rules.

MainActivity.kt · lines 182–211

```
182     CompositionLocalProvider(LocalContentColor provides Color.White) {
183         Box(
184             modifier = modifier
```

```
185                 .fillMaxSize()
186                 .background(Brush.verticalGradient(listOf(NightTop, NightMiddle, NightBottom)))
187                 .drawBehind {
188                     // Two soft colored glows behind the content.
189                     val pinkCenter = Offset(0f, size.height * 0.2f)
190                     val blueCenter = Offset(size.width, size.height * 0.75f)
191                     val radius = size.width * 0.8f
192                     drawCircle(
193                         brush = Brush.radialGradient(
194                             listOf(LeftCardStart.copy(alpha = 0.3f), Color.Transparent),
195                             center = pinkCenter,
196                             radius = radius
197                         ),
198                         radius = radius,
199                         center = pinkCenter
200                     )
201                     drawCircle(
202                         brush = Brush.radialGradient(
203                             listOf(RightCardStart.copy(alpha = 0.25f), Color.Transparent),
204                             center = blueCenter,
205                             radius = radius
206                         ),
207                         radius = radius,
208                         center = blueCenter
209                     )
210                 }
211         ) {
```

- **182**: Makes white the default color for everything inside. Any `Text` without its own `color` is white. `provides` pairs the setting with its value.
- **183**: `Box` stacks its children on top of each other. This outer Box holds the background, the game and the pop-up.
- **184**: Starts from the `modifier` passed in at line 134 (lowercase means the parameter) and adds to it.
- **185**: Fills the whole screen.
- **186**: The background: a top-to-bottom gradient of three dark colors. `listOf(...)` makes a list.
- **187**: `drawBehind { }` lets you draw shapes with code. Modifiers apply in order, so this draws *on top of* the gradient from line 186 but *behind* the Box's children. Inside it, `size` is the Box's size in pixels.
- **189**: `Offset(x, y)` is a point. The pink glow's center is on the left edge (x = 0), 20% of the way down.
- **190**: The blue glow's center is on the right edge (x = full width), 75% of the way down.
- **191**: Both glows have a radius of 80% of the screen width.
- **192–200**: Draws the pink glow.
  - Its brush is a radial gradient: pink in the middle, fading to transparent at the edge.
  - `.copy(alpha = 0.3f)` means the same color at 30% opacity.
  - Because the center is on the left edge, only half the circle is visible. It looks like light coming from the side.
- **201–209**: The same for the blue glow, at 25% opacity, in the bottom right.
- **211**: `) {` closes the Box's settings and opens its content.

Lines 212–226: Safe area and scrolling

MainActivity.kt · lines 212–226

```
212             BoxWithConstraints(
213                 modifier = Modifier
214                     .fillMaxSize()
215                     .safeDrawingPadding()
216             ) {
217                 // Fills the screen in portrait, and scrolls if it doesn't fit (e.g. landscape).
218                 Column(
219                     modifier = Modifier
220                         .fillMaxWidth()
221                         .verticalScroll(rememberScrollState())
222                         .heightIn(min = maxHeight)
223                         .padding(horizontal = 20.dp, vertical = 16.dp),
224                     horizontalAlignment = Alignment.CenterHorizontally,
225                     verticalArrangement = Arrangement.SpaceBetween
226                 ) {
```

- **212**: `BoxWithConstraints` is a Box that tells its content how much space it has (`maxWidth`, `maxHeight`).
- **213**: Capital `Modifier` means a fresh, empty modifier. The caller's `modifier` was already used on the outer Box.
- **214**: Fills the screen.
- **215**: Adds padding equal to the size of the status bar, navigation bar and camera cutout.
  - It's needed because of edge-to-edge (line 106). Without it, the title would sit under the clock.
  - The background is on the outer Box, which has no padding, so it still reaches the screen edges.
- **218**: `Column` places its children top to bottom.
- **220**: Full width.
- **221**: Makes the Column scroll up and down. `rememberScrollState()` remembers the scroll position.
- **222**: Makes the Column at least as tall as the visible area (`maxHeight` comes from line 212). Inside a scroll, a Column would otherwise only be as tall as its content, and line 225 couldn't push things to the top and bottom.
- **223**: 20 dp of space on the left and right, 16 dp at the top and bottom.
- **224**: Centers the children horizontally.
- **225**: `SpaceBetween` puts the first child at the top, the last child at the bottom, and spreads the rest in between. That puts the header at the top, the game in the middle and the score at the bottom.

The result: in portrait everything fits exactly. In landscape it doesn't fit, so you can scroll.

Lines 227–268: The three sections

MainActivity.kt · lines 227–268

```
227                     GameHeader()
228
229                     Column(
230                         modifier = Modifier.padding(vertical = 24.dp),
231                         horizontalAlignment = Alignment.CenterHorizontally
232                     ) {
233                         Text(
234                             text = stringResource(R.string.which_is_bigger),
235                             style = MaterialTheme.typography.titleMedium,
236                             fontWeight = FontWeight.Bold
237                         )
238                         Spacer(Modifier.height(20.dp))
```

```
239                         Row(
240                             modifier = Modifier
241                                 .fillMaxWidth()
242                                 .graphicsLayer { translationX = shake.value.dp.toPx() },
243                             horizontalArrangement = Arrangement.spacedBy(12.dp),
244                             verticalAlignment = Alignment.CenterVertically
245                         ) {
246                             NumberCard(
247                                 number = leftNumber,
248                                 round = round,
249                                 colors = listOf(LeftCardStart, LeftCardEnd),
250                                 onClick = { onGuess(pickedLeft = true) },
251                                 modifier = Modifier.weight(1f)
252                             )
253                             VsBadge()
254                             NumberCard(
255                                 number = rightNumber,
256                                 round = round,
257                                 colors = listOf(RightCardStart, RightCardEnd),
258                                 onClick = { onGuess(pickedLeft = false) },
259                                 modifier = Modifier.weight(1f)
260                             )
261                         }
262                         Spacer(Modifier.height(20.dp))
263                         FeedbackMessage(round = round, lastGuessCorrect = lastGuessCorrect)
264                     }
265 
266                     ScoreCard(points = points)
267                 }
268             }
```

- **227**: Child 1 of the column (top): the title and instructions (lines 322–341).
- **229**: Child 2 (middle): the question, the cards and the message.
- **230**: 24 dp of space above and below.
- **233–237**: The question.
  - **234**: `stringResource(R.string.which_is_bigger)` loads “Which number is bigger?” from `strings.xml`.
  - **235**: A medium title style from the theme.
  - **236**: Bold. It’s white because of line 182.
- **238**: A 20 dp gap.
- **239**: `Row` places its children left to right.
- **241**: Full width.
- **242**: **The shake.**
  - `graphicsLayer` changes how the row is *drawn* without changing the layout.
  - `translationX` is a sideways shift, in pixels.
  - `shake.value` is in dp (for example −24). `.dp` makes it a dp value, and `.toPx()` converts it to pixels for this screen.
  - Because the value is read inside `graphicsLayer`, Compose only redraws the row each frame instead of rebuilding anything, which keeps it smooth.
- **243**: A 12 dp gap between the children.
- **244**: Centers the children vertically, so the 44 dp VS circle sits in the middle of the 170 dp cards.
- **246–252**: The left card.
  - **247**: Shows `leftNumber`.

- **248**: Passes `round`, which is used for the animation.
- **249**: A pink-to-red gradient.
- **250**: **This connects the tap to the rules.** `{ onGuess(pickedLeft = true) }` is a *lambda*, a function passed as a value. The card runs it when it's tapped.
- **251**: `weight(1f)` gives each card one equal share of the width left over after the VS circle. `weight` only works inside a Row or Column.

- **253**: The VS circle.
- **254–260**: The right card. It's the same, but with `rightNumber`, the blue gradient and `pickedLeft = false`.
- **262**: A 20 dp gap.
- **263**: The Correct!/Wrong! message, given the round and the last result.
- **266**: Child 3 (bottom): the score card.
- **267–268**: Close the scrolling Column and the `BoxWithConstraints`.

Lines 270–320: The pop-ups

MainActivity.kt · lines 270–320

```
270             when (dialog) {
271                 GameDialog.Genius -> FancyDialog(
272                     emoji = "🏆",
273                     title = stringResource(R.string.genius_title),
274                     message = stringResource(R.string.genius_message, GENIUS_SCORE),
275                     gradient = listOf(GoldStart, GoldEnd),
276                     textColor = GeniusText,
277                     // Back button or tapping outside also starts a new game, same as Play Again.
278                     onDismissRequest = { playAgain() }
279                 ) {
280                     Button(
281                         onClick = { playAgain() },
282                         colors = ButtonDefaults.buttonColors(
283                             containerColor = GeniusText,
284                             contentColor = Color.White
285                         ),
286                         modifier = Modifier
287                             .fillMaxWidth()
288                             .height(52.dp)
289                     ) {
290                         Text(stringResource(R.string.play_again), fontSize = 16.sp, fontWeight =
    FontWeight.Bold)
291                     }
292                 }
293
294                 GameDialog.Dumb -> FancyDialog(
295                     emoji = "🤦",
296                     title = stringResource(R.string.dumb_title),
297                     message = stringResource(R.string.dumb_message),
298                     gradient = listOf(DumbStart, DumbEnd),
299                     textColor = Color.White,
300                     onDismissRequest = { dialog = null }
301                 ) {
302                     Button(
303                         onClick = { dialog = null },
304                         colors = ButtonDefaults.buttonColors(
305                             containerColor = Color.White,
306                             contentColor = DumbStart
307                         ),
308                         modifier = Modifier
309                             .fillMaxWidth()
```

```
310                             .height(52.dp)
311                     ) {
312                         Text(stringResource(R.string.try_again), fontSize = 16.sp, fontWeight =
    FontWeight.Bold)
313                     }
314                 }
315
316                 null -> Unit
317             }
318         }
319     }
320 }
```

- **270**: `when (dialog)` works like a `switch`. It checks which pop-up should show every time the screen redraws. It's written inside the outer Box, but `Dialog` opens in its own window on top of the app, so where it sits in the code doesn't matter.
- **271**: If `dialog` is `Genius`, draw a `FancyDialog` with:
  - **272**: a trophy emoji,
  - **273**: the title "✨ GENIUS! ✨",
  - **274**: the message, with the `%1$d` from `strings.xml` replaced by 10,
  - **275**: a gold gradient,
  - **276**: dark brown text, which is readable on gold.
- **278**: What happens when the user presses Back or taps outside the pop-up: the same as Play Again.
- **279**: The `{ }` after the `)` is a *trailing lambda*. It's passed as the `actions` parameter (line 507), which is the button at the bottom of the pop-up.
- **280–281**: A Material 3 button. Tapping it calls `playAgain()`, which sets `dialog = null`, so the pop-up disappears and the score resets.
- **282–285**: The button's colors: dark brown background, white text. `ButtonDefaults.buttonColors(...)` keeps the default colors for anything you don't set, such as the disabled state.
- **286–288**: Full width, 52 dp tall.
- **290**: The label "Play Again", 16 sp, bold.
- **294–314**: The Dumb pop-up has the same structure, with these differences:
  - a 🟥🟩🟦 emoji, a red gradient and white text,
  - a white button with red text (305–306),
  - Back, tapping outside and Try Again only run `dialog = null` (300, 303). That just closes the pop-up; the points stay at 0 and you keep playing.
- **316**: `null -> Unit`: there's no pop-up, so draw nothing. `Unit` is Kotlin's "nothing" value. Kotlin requires this `when` to handle every possible value of `dialog`: Genius, Dumb *and* null.
- **318–320**: Close the Box, the white-text provider, and the function.

Lines 322–341: GameHeader

MainActivity.kt · lines 322–341

```
@Composable
private fun GameHeader() {
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(
            text = stringResource(R.string.game_title),
            style = MaterialTheme.typography.headlineMedium.copy(
                brush = Brush.horizontalGradient(listOf(GoldEnd, LeftCardStart))
            ),
            fontWeight = FontWeight.Black,
            textAlign = TextAlign.Center
        )
        Spacer(Modifier.height(8.dp))
        Text(
            text = stringResource(R.string.game_instructions),
            style = MaterialTheme.typography.bodyMedium,
            color = Color.White.copy(alpha = 0.75f),
            textAlign = TextAlign.Center
        )
    }
}
```

- **323**: `private` means only this file can use it. It has no parameters because it always shows the same thing.
- **324**: A centered column.
- **326**: “Bigger Number Game!”.
- **327**: Takes the theme’s large `headlineMedium` style and makes a `.copy(...)` of it with one change:
- **328**: The letters are filled with a yellow-to-pink gradient. A brush overrides the white color from line 182.
- **330**: `FontWeight.Black` is the heaviest weight, thicker than Bold.
- **331**: Centers the lines, which matters when the text wraps onto two lines.
- **333**: An 8 dp gap.
- **335–338**: The instructions, in a smaller body style. They’re white at 75% opacity, so they look less important than the title, and centered.

Lines 343–390: NumberCard

MainActivity.kt · lines 343–390

```
/** A big gradient card showing one number. It shrinks a little while pressed. */
@Composable
private fun NumberCard(
    number: Int,
    round: Int,
    colors: List<Color>,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    val interactionSource = remember { MutableInteractionSource() }
    val pressed by interactionSource.collectIsPressedAsState()
    val scale by animateFloatAsState(if (pressed) 0.92f else 1f, label = "pressScale")
    val shape = RoundedCornerShape(28.dp)

    Surface(
        onClick = onClick,
        modifier = modifier
            .height(170.dp)
```

```
            .graphicsLayer {
                scaleX = scale
                scaleY = scale
            }
            .shadow(elevation = 16.dp, shape = shape, spotColor = colors.last()),
        shape = shape,
        color = Color.Transparent,
        contentColor = Color.White,
        interactionSource = interactionSource
    ) {
        Box(
            modifier = Modifier
                .fillMaxSize()
                .background(Brush.linearGradient(colors)),
            contentAlignment = Alignment.Center
        ) {
            // Keyed on the round too, so the number pops in every round, even if it didn't change.
            AnimatedContent(
                targetState = round to number,
                transitionSpec = {
                    (scaleIn(initialScale = 0.5f) + fadeIn()) togetherWith
                        (scaleOut(targetScale = 1.5f) + fadeOut())
                },
                label = "number"
            ) { (_, value) ->
                Text(text = value.toString(), fontSize = 72.sp, fontWeight = FontWeight.Black)
            }
        }
    }
}
```

- **346–350**: The parameters:
  - `number`: the number to show.
  - `round`: the current round, used for the animation.
  - `colors`: the gradient colors. This is how one function makes both the pink card and the blue card.
  - `onClick: () -> Unit`: a function that takes nothing and returns nothing. The card calls it when it's tapped, without knowing anything about the rules. This pattern is called *state hoisting*: the parent owns the data and the rules, and the card only shows things and reports taps.
  - `modifier`: comes from the parent; here it's `Modifier.weight(1f)`.
- **352**: A stream of touch events (press, release). `remember` creates it once and keeps it.
- **353**: Turns that stream into a Boolean state that is `true` while a finger is on the card.
- **354**: `animateFloatAsState` returns a value that moves smoothly toward its target: 0.92 (92% size) while pressed, 1.0 when released. `label` is just a name shown in Android Studio's animation tools.
- **355**: A rectangle with 28 dp rounded corners. It's stored in a `val` because it's used twice.
- **357**: A Material 3 `Surface` with `onClick` is a tappable container with the ripple effect. It also clips its content to `shape`.
- **358**: When the card is tapped, this calls the function from the parent, which is `onGuess`.
- **359**: Starts from the parent's modifier (the weight), then adds the next three lines:
- **360**: The card is 170 dp tall. Its width comes from the weight.
- **361–364**: Scales the drawing by `scale`, so the card shrinks to 92% while pressed. Because it uses `graphicsLayer`, the layout doesn't change and the other card doesn't move.
- **365**: A 16 dp shadow in the same rounded shape.
  - It's colored with the last gradient color, which gives a red or blue glow.

    - Colored shadows need Android 9 or newer; older phones show a grey one.
    - It comes after the scale in the chain, so the shadow shrinks with the card.
- **366**: The Surface's shape, used for the rounded corners and the clipping.
- **367**: Transparent, because the Surface has no color of its own. The gradient Box inside paints it.
- **368**: The default content color is white, for the number.
- **369**: Gives the Surface our `interactionSource`, so the presses it detects are the same ones line 353 watches.
- **371–375**: A Box that fills the card, with a diagonal gradient and centered content. `linearGradient` goes from the top-left to the bottom-right by default.
- **378**: `AnimatedContent`: whenever `targetState` changes, it animates from the old content to the new.
- **379**: `round to number` makes a pair, `(round, number)`.
    - With `number` alone, a new round that happened to show the same number (say 7 again) would change nothing. There'd be no animation, and the tap would look like it did nothing.
    - `round` changes every round, so the pair always changes.
- **380–383**: The transition. The new number grows in from 50% while fading in. At the same time (`togetherWith`), the old number grows to 150% while fading out. `+` combines two effects.
- **385**: The lambda receives the pair and unpacks it: `_` ignores the round, and `value` is the number. During the animation, the old and new numbers are drawn at the same time, each with its own `value`. That's why it uses `value` here and not `number`.
- **386**: Draws the number at 72 sp in the heaviest weight. `toString()` turns the Int into text.

Lines 392–402: VsBadge

MainActivity.kt · lines 392–402

```
392 @Composable
393 private fun VsBadge() {
394     Box(
395         modifier = Modifier
396             .size(44.dp)
397             .glass(CircleShape),
398         contentAlignment = Alignment.Center
399     ) {
400         Text(text = stringResource(R.string.vs), fontSize = 14.sp, fontWeight = FontWeight.Black)
401     }
402 }
```

- **396**: 44 × 44 dp.
- **397**: The `glass` helper (line 571): a circle with a faint white fill and a thin border.
- **398**: Centers the content.
- **400**: "VS", 14 sp, heaviest weight, white because of line 182.

Lines 404–431: FeedbackMessage

MainActivity.kt · lines 404–431

```
404 /** Shows "Correct!" or "Wrong!" after each guess, popping in again every round. */
405 @Composable
406 private fun FeedbackMessage(round: Int, lastGuessCorrect: Boolean?) {
407     AnimatedContent(
408         targetState = round to lastGuessCorrect,
409         transitionSpec = { (scaleIn(initialScale = 0.7f) + fadeIn()) togetherWith fadeOut() },
410         label = "feedback"
```

```
411     ) { (_, correct) ->
412         Box(modifier = Modifier.height(48.dp), contentAlignment = Alignment.Center) {
413             if (correct == null) {
414                 Text(
415                     text = stringResource(R.string.tap_to_start),
416                     color = Color.White.copy(alpha = 0.7f)
417                 )
418             } else {
419                 Text(
420                     text = stringResource(if (correct) R.string.correct else R.string.incorrect),
421                     fontSize = 16.sp,
422                     fontWeight = FontWeight.Bold,
423                     modifier = Modifier
424                         .clip(CircleShape)
425                         .background(if (correct) CorrectGreen else WrongRed)
426                         .padding(horizontal = 20.dp, vertical = 10.dp)
427                 )
428             }
429         }
430     }
431 }
```

- **406**: Takes the round and the last result. `Boolean?` has three possible values: null, true, false.
- **408**: A pair again, so two "Correct!" messages in a row still animate.
- **409**: The new message grows in from 70% while fading in. The old one just fades out.
- **411**: Unpacks the pair, so `correct` is the `Boolean?`.
- **412**: A Box with a fixed height of 48 dp and centered content. The fixed height stops the cards and the score from jumping when the message changes.
- **413–417**: No guess yet: shows "Tap the bigger number to start!" in 70% white.
- **418**: Otherwise (true or false):
- **420**: Picks the string: "✅ Correct!" for true, "❌ Wrong!" for false. Inside the `else`, Kotlin knows `correct` isn't null (this is called a *smart cast*), so `if (correct)` is allowed.
- **421–422**: 16 sp, bold.
- **424**: Clips to `CircleShape`. On a wide box, that gives a pill shape with fully rounded ends.
- **425**: Green or red background.
- **426**: Space inside the pill, around the text. The order matters: clip → background → padding means the color covers the padding too. If padding came first, the color would only cover the text.

Lines 433–496: ScoreCard

MainActivity.kt · lines 433–496

```
433 /** The points display, with a progress bar towards the Genius score. */
434 @Composable
435 private fun ScoreCard(points: Int) {
436     val progress by animateFloatAsState(
437         targetValue = points.coerceIn(0, GENIUS_SCORE) / GENIUS_SCORE.toFloat(),
438         label = "progress"
439     )
440 
441     Column(
442         modifier = Modifier
443             .fillMaxWidth()
444             .glass(RoundedCornerShape(24.dp))
445             .padding(20.dp),
446         horizontalAlignment = Alignment.CenterHorizontally
```

```
    ) {
        Text(
            text = stringResource(R.string.points_label),
            fontSize = 12.sp,
            fontWeight = FontWeight.Bold,
            letterSpacing = 4.sp,
            color = Color.White.copy(alpha = 0.7f)
        )
        // The number slides up when points go up, and down when they go down.
        AnimatedContent(
            targetState = points,
            transitionSpec = {
                if (targetState > initialState) {
                    (slideInVertically { it } + fadeIn()) togetherWith (slideOutVertically { -it } +
fadeOut())
                } else {
                    (slideInVertically { -it } + fadeIn()) togetherWith (slideOutVertically { it } +
fadeOut())
                }.using(SizeTransform(clip = false))
            },
            label = "points"
        ) { value ->
            Text(text = value.toString(), fontSize = 48.sp, fontWeight = FontWeight.Black)
        }
        Spacer(Modifier.height(8.dp))
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .height(10.dp)
                .clip(CircleShape)
                .background(Color.White.copy(alpha = 0.15f))
        ) {
            Box(
                modifier = Modifier
                    .fillMaxWidth(progress)
                    .fillMaxHeight()
                    .clip(CircleShape)
                    .background(Brush.horizontalGradient(listOf(GoldStart, GoldEnd)))
            )
        }
        Spacer(Modifier.height(6.dp))
        Text(
            text = stringResource(
                R.string.genius_progress,
                points.coerceAtMost(GENIUS_SCORE),
                GENIUS_SCORE
            ),
            fontSize = 12.sp,
            color = Color.White.copy(alpha = 0.7f)
        )
    }
}
```

- **435**: Only needs `points`.
- **436–439**: `progress` is a number from 0.0 to 1.0, animated so the bar grows smoothly.
  - **437**: `coerceIn(0, GENIUS_SCORE)` keeps the points between 0 and 10, as a safety check. Dividing by `GENIUS_SCORE.toFloat()` gives, for example, 3 / 10f = 0.3. The `.toFloat()` is essential, because dividing an Int by an Int drops the decimals: 3 / 10 would be 0.
- **441–446**: The card: full width, the glass look with 24 dp rounded corners, 20 dp of space inside, and centered content.

- **448–454**: The “POINTS” label: 12 sp, bold, letters spread 4 sp apart, 70% white.
- **456–457**: Animates the number whenever `points` changes. There’s no `round` here on purpose: when the points don’t change (a wrong answer at 0), there should be no animation.
- **458**: Inside `transitionSpec`, `initialState` is the old value and `targetState` is the new one.
- **459–460**: Going up: the new number slides in from below and the old one slides out upward, both fading. In `{ it }`, `it` is the content’s height, so a positive value means “below”.
- **461–462**: Going down: the same, in the opposite directions.
- **463**: `SizeTransform` controls what happens when the old and new content are different sizes (for example, 10 is wider than 9). `clip = false` means the sliding numbers aren’t cut off. Because `if/else` returns a value, `.using(...)` applies to whichever transition was chosen.
- **466–467**: `value` is the points for each piece of content (old or new), drawn at 48 sp.
- **469**: An 8 dp gap.
- **470–476**: The outer Box is the **track**: full width, 10 dp tall, pill-shaped, 15% white.
- **477–483**: The inner Box is the **gold fill**.
  - `fillMaxWidth(progress)` makes it `progress` × the track’s width, so 0.3 covers 30%.
  - It’s the full height of the track, pill-shaped, with an orange-to-yellow gradient.
- **485**: A 6 dp gap.
- **486–494**: The text “3 / 10 to Genius”.
  - The extra values passed to `stringResource` fill in `%1$d` and `%2$d`.
  - `coerceAtMost(GENIUS_SCORE)` never shows more than 10.
  - 12 sp, 70% white.

Lines 498–568: FancyDialog

MainActivity.kt · lines 498–568

```
498 /** A colorful pop-up that bounces in, with a pulsing emoji on top. */
499 @Composable
500 private fun FancyDialog(
501     emoji: String,
502     title: String,
503     message: String,
504     gradient: List<Color>,
505     textColor: Color,
506     onDismissRequest: () -> Unit,
507     actions: @Composable ColumnScope.() -> Unit
508 ) {
509     Dialog(onDismissRequest = onDismissRequest) {
510         val popIn = remember { Animatable(0.6f) }
511         LaunchedEffect(Unit) {
512             popIn.animateTo(
513                 targetValue = 1f,
514                 animationSpec = spring(
515                     dampingRatio = Spring.DampingRatioMediumBouncy,
516                     stiffness = Spring.StiffnessMediumLow
517                 )
518             )
519         }
520         val pulse by rememberInfiniteTransition(label = "pulse").animateFloat(
521             initialValue = 1f,
522             targetValue = 1.15f,
523             animationSpec = infiniteRepeatable(tween(durationMillis = 700), RepeatMode.Reverse),
524             label = "emojiScale"
```

```
525         )
526         val shape = RoundedCornerShape(28.dp)
527 
528         CompositionLocalProvider(LocalContentColor provides textColor) {
529             Column(
530                 modifier = Modifier
531                     .graphicsLayer {
532                         scaleX = popIn.value
533                         scaleY = popIn.value
534                         alpha = ((popIn.value - 0.6f) / 0.4f).coerceIn(0f, 1f)
535                     }
536                     .shadow(elevation = 24.dp, shape = shape)
537                     .clip(shape)
538                     .background(Brush.linearGradient(gradient))
539                     .padding(28.dp),
540                 horizontalAlignment = Alignment.CenterHorizontally
541             ) {
542                 Text(
543                     text = emoji,
544                     fontSize = 72.sp,
545                     modifier = Modifier.graphicsLayer {
546                         scaleX = pulse
547                         scaleY = pulse
548                     }
549                 )
550                 Spacer(Modifier.height(12.dp))
551                 Text(
552                     text = title,
553                     style = MaterialTheme.typography.headlineSmall,
554                     fontWeight = FontWeight.Black,
555                     textAlign = TextAlign.Center
556                 )
557                 Spacer(Modifier.height(8.dp))
558                 Text(
559                     text = message,
560                     style = MaterialTheme.typography.bodyLarge,
561                     textAlign = TextAlign.Center
562                 )
563                 Spacer(Modifier.height(24.dp))
564                 actions()
565             }
566         }
567     }
568 }
```

- **501–505**: What the pop-up shows: the emoji, title and message, the gradient colors, and the text color.
- **506**: What to run when the user presses Back or taps outside.
- **507**: The button area. The type `@Composable ColumnScope.() -> Unit` means “a block of UI that runs inside a Column and returns nothing”. That’s how the callers pass in `{ Button(...) }` (lines 279 and 301).
- **509**: `Dialog` opens a new window above the app with a dimmed background, and it blocks taps on the game behind it. It does **not** close itself; it only calls `onDismissRequest`. The pop-up disappears when the caller sets `dialog = null`.
- **510**: The pop-in scale starts at 0.6 (60%). It’s remembered inside the dialog, so every new pop-up starts at 0.6 again.
- **511**: `LaunchedEffect(Unit)` runs its block in a coroutine once, when the pop-up first appears. `Unit` never changes, so the block never restarts. A coroutine is needed because `animateTo` waits for the animation to finish.
- **512–518**: Animates the scale to 1.0 with a spring (physics-based motion):

    - **515**: `DampingRatioMediumBouncy` makes it go a little past 1.0 and settle back. That's the bounce.
    - **516**: `StiffnessMediumLow` makes the spring softer and slower.
- **520**: An animation that keeps running as long as the pop-up is open. `by` makes `pulse` a plain Float.
- **521–522**: It goes between 1.0 and 1.15.
- **523**: `tween(700)` takes 700 ms in each direction. `RepeatMode.Reverse` makes it go back and forth (1 → 1.15 → 1 → …) instead of jumping straight back to 1.
- **526**: The card's rounded shape.
- **528**: The default text color inside becomes `textColor` (brown or white). The button sets its own text color, so it isn't affected.
- **529**: A column holding the emoji, title, message and button.
- **531–535**: Reads `popIn.value` on every frame:
    - **532–533**: The scale follows `popIn`: 0.6 → slightly over 1 → 1.0.
    - **534**: The opacity. `(value - 0.6) / 0.4` turns 0.6 into 0 and 1.0 into 1, so 0.8 gives 0.5. The pop-up fades in while it grows. `coerceIn(0f, 1f)` caps it when the spring goes past 1, because opacity can't be above 1.
- **536**: A large 24 dp shadow.
- **537**: Clips to the rounded shape, so the gradient gets rounded corners.
- **538**: The gold or red gradient background.
- **539**: 28 dp of space inside.
- **540**: Centers everything.
- **542–549**: The emoji at 72 sp, scaled by `pulse` to make the heartbeat effect.
- **550, 557, 563**: Gaps of 12, 8 and 24 dp.
- **551–556**: The title: `headlineSmall` style, heaviest weight, centered.
- **558–562**: The message: `bodyLarge` style, centered.
- **564**: `actions()` runs the block the caller passed in, which draws the Play Again or Try Again button here.

Lines 570–574: The glass helper

MainActivity.kt · lines 570–574

```
570 /** Frosted-glass look: a faint white fill with a thin white border. */
571 private fun Modifier.glass(shape: Shape): Modifier = this
572     .clip(shape)
573     .background(Color.White.copy(alpha = 0.08f))
574     .border(1.dp, Color.White.copy(alpha = 0.18f), shape)
```

- **571**: An *extension function*. It adds a `.glass(...)` method to `Modifier`, so you can use it in a chain like any built-in modifier. `= this …` returns the chain so far (`this`) plus three more modifiers. It isn't `@Composable`; it only returns a Modifier.
- **572**: Clips to the shape.
- **573**: An 8% white fill.
- **574**: A 1 dp border, 18% white, that follows the shape.

It's used by `VsBadge` (a circle) and `ScoreCard` (a rounded rectangle).

Lines 576–582: Preview

MainActivity.kt · lines 576–582

```
576 @Preview(showBackground = true)
577 @Composable
578 fun BiggerNumberGamePreview() {
579     Cse489ProjectTheme {
580         BiggerNumberGame()
581     }
582 }
```

- **576**: Tells Android Studio to show this function in the **Split/Design** view. `showBackground = true` draws a white background behind it.
- **577–582**: Shows the game inside the theme. The real app never calls this; it exists only for the preview.

Part 3: Follow some taps through the code

Example 1: a correct guess. Left = 7, right = 3, points = 2, round = 5. You tap the left card:

1. The Surface's `onClick` (358) runs `{ onGuess(pickedLeft = true) }` (250).
2. **155:** 7 > 3, so `correct = true`. **156:** `lastGuessCorrect = true`.
3. **159:** a success vibration. **160:** `points = 3`. **161:** 3 isn't 10, so no pop-up.
4. **172** → **148–150:** new numbers, say left 4 and right 8, and `round = 6`.
5. The state changed, so Compose redraws:
   - **NumberCard (379):** the pairs changed from (5, 7) and (5, 3) to (6, 4) and (6, 8). The 7 and 3 grow and fade out while the 4 and 8 grow in.
   - **FeedbackMessage (408):** the pair is now (6, true), so a green "✅ Correct!" pops in.
   - **ScoreCard:** 2 → 3, so the number slides up (460), the bar animates from 20% to 30% (437), and the text says "3 / 10 to Genius".

Example 2: a wrong guess at 0 points. Left = 2, right = 9, points = 0. You tap the left card:

1. **155:** 2 > 9 is false, so `correct = false`.
2. **163:** an error vibration. **164–166:** the cards shake for 300 ms.
3. **170:** `points == 0`, so `dialog = GameDialog.Dumb` and the points stay at 0.
4. **172:** new numbers are picked behind the pop-up.
5. The screen redraws:
   - a red "❌ Wrong!" appears,
   - the score doesn't animate, because it's still 0,
   - **294:** the Dumb pop-up bounces in (510–518) with a pulsing 🥵 (520–523).
6. You tap **Try Again**. **303:** `dialog = null`, so **316** draws nothing. The pop-up is gone and you keep playing.

Example 3: winning. At 9 points, a correct guess makes **160** set `points = 10`. Then **161** sets `dialog = GameDialog.Genius`, and the 🏆 pop-up appears. Tapping **Play Again** runs `playAgain()` (175–180): points go back to 0, the message resets, the pop-up closes, and there are new numbers.